

A. CARUSO!

# Sistemi Operativi 1

prof: Finzi A.

**CONSIGLI:** I fondamenti teorici non sono Totalmente necessari per svolgere gli esercizi della prova scritta, il prof durante la prova scritta vede che si risponde CORRETTAMENTE ad almeno 1 domanda orale su 3, prova scritta sono 7 esercizi e 3 domande orali.

**CONSIGLI per la prova orale:** Non consiglio gli appunti di lezione, perché descrive tutti gli argomenti trattati, ma in modo confuso e poco semplice da interpretare, personalmente mi sono trovato meglio con le videolezioni caricate su youtube di Walter Balzano, spiegazioni semplici ed esempi molto intuitivi.

**CANALE YOUTUBE:**

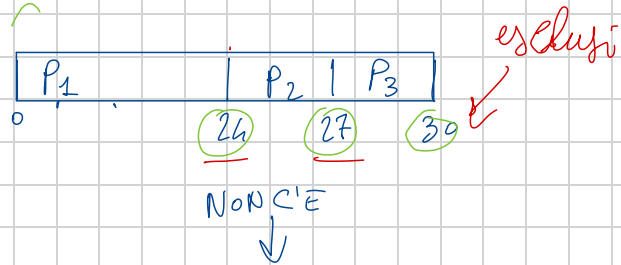
[https://youtube.com/@videolezioni8404?  
si=JAMEUFWITX1PR8vc](https://youtube.com/@videolezioni8404?si=JAMEUFWITX1PR8vc)

**Avviso:** Le informazioni contenute in questa pagina sono frutto delle mie riflessioni e delle discussioni avute con altri colleghi durante la preparazione dell'esame. Non rappresentano una verità assoluta, ma piuttosto un punto di vista che potrebbe esserti utile come spunto di riflessione.

# Scheduling Processi <sup>code P<sub>2</sub> P<sub>3</sub></sup> $P_1 \rightarrow$ CPU

★ **FCFS** il primo che arriva viene servito

| Process        | Burst Time | AT |
|----------------|------------|----|
| P <sub>1</sub> | 24         | 0  |
| P <sub>2</sub> | 3          | 1  |
| P <sub>3</sub> | 3          | 2  |



★ temp. attesa = t. completamento - t. di arrivo - Burst Time

★ temp. risp. = t. inizio esecuzione - t. arrivo

★ tourn around = t. complet. - t. arrivo

★ tourn around = t. attesa + burst time

t attesa per processo

NON HAI Arrival Time

$$P_1 (24 - 24) = 0 \leftarrow +$$

$$P_2 (27 - 3) = 24 \leftarrow +$$

$$P_3 (30 - 3) = 27 \leftarrow = \frac{17 \text{ ms}}{3} \text{ t. attesa medio}$$

$$\text{tourn around} = P_1 \quad 0 + 24 = 24$$

$$P_2 \quad 24 + 3 = 27$$

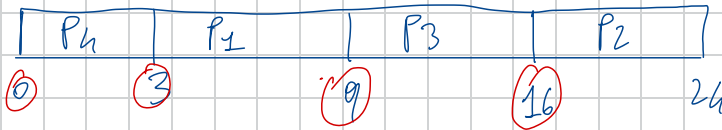
$$P_3 \quad 27 + 3 = 30$$

$\swarrow$   
n processi

★ SJF senza prelazione

← considera il job meno impegnativo per la CPU, quindi quello con meno Burst time

| Process | Burst |    |   |
|---------|-------|----|---|
| $P_1$   | 6     | 2° | ✓ |
| $P_2$   | 8     | 4° | ✓ |
| $P_3$   | 7     | 3° | ✓ |
| $P_4$   | 3     | 1° | ✓ |



$$T_{attesa} = P_1 (9 - 6) \\ P_2 (24 - 8) \\ P_3 (16 - 7) \\ P_4 (3 - 3)$$

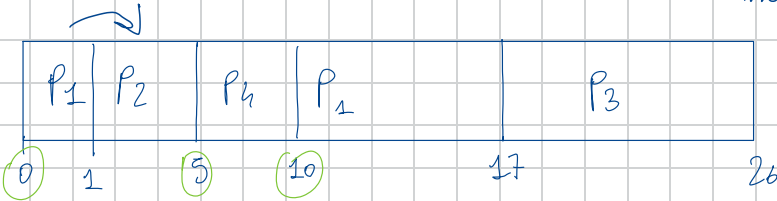


★ SJF predatione = SRTF il  $P_2$  parte perché è l'unico processo in code, quando arriva  $P_2$  si

| Process | Burst          | Arrival t. |
|---------|----------------|------------|
| $P_1$   | <del>8</del> 7 | 0          |
| $P_2$   | <del>4</del> ✓ | 1          |
| $P_3$   | 9              | 2          |
| $P_4$   | (5)            | 3          |

confronta il Burst rimanente di  $P_1$  (8-1), (perché  $P_2$  arriva all'istante 1 quindi  $P_1$  ha già eseguito 1 unità di tempo) con il Burst t. di  $P_2$ , se  $P_2$  ha un tempo di esecuzione minore,  $P_1$  viene messo in ready queue e

viene sostituito da  $P_2$ .



code

$P_1(7)$

$P_3(9)$

$$t_{resp} = P_1(0 - 0) = 0$$

$$P_2(1 - 1) = 0$$

$$P_3(17 - 2) = 15$$

$$P_4(5 - 3) = 2$$

$$t_{attest} = P_1(17 - 0 - 8) = 9$$

$$P_2(5 - 1 - 4) = 0$$

$$P_3(26 - 2 - 9) = 15$$

$$P_4(10 - 3 - 5) = 2$$


---


$$26/4 = 6,5 \text{ ms}$$

$$t_{resp. medio} = 17/4 = 4,25 \text{ ms}$$

turn around = completion - arrival

$$P_1(17 - 0) = 17$$

$$P_2(5 - 1) = 4$$

$$P_3(26 - 2) = 24$$

$$P_4(10 - 3) = 7$$

---


$$52/4 = 13 \text{ ms}$$

turn ar. = attest + burst

$$P_1(9 + 8) = 17$$

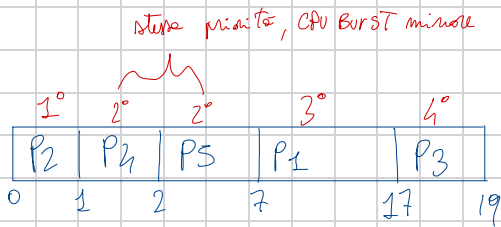
$$P_2(0 + 4) = 4$$

$$P_3(15 + 9) = 24$$

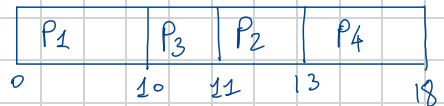
$$P_4(2 + 5) = 7$$

★ SJF priorità: il confronto si può fare solo con processi di stessa priorità.

| Process        | Burst | Priority |
|----------------|-------|----------|
| P <sub>1</sub> | 10    | 3°       |
| P <sub>2</sub> | 1     | 1°       |
| P <sub>3</sub> | 2     | 4°       |
| P <sub>4</sub> | 1     | 2°       |
| P <sub>5</sub> | 5     | 2°       |

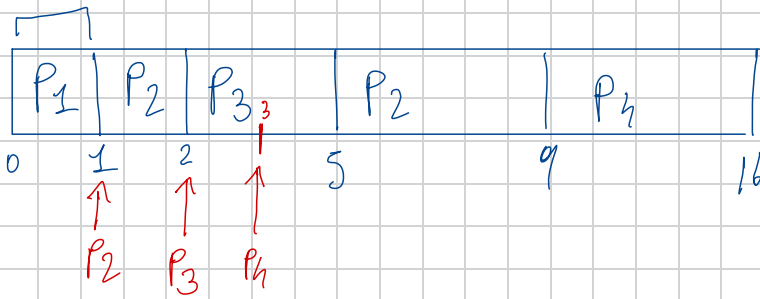


| Process        | Burst | Priority |
|----------------|-------|----------|
| P <sub>1</sub> | 10    | 1°       |
| P <sub>2</sub> | 2     | 2°       |
| P <sub>3</sub> | 1     | 2°       |
| P <sub>4</sub> | 5     | 3°       |



★ SJF priorità e preemption

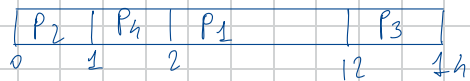
| Processi       | Burst | Arrival time | Priority |
|----------------|-------|--------------|----------|
| P <sub>1</sub> | 1     | 0            | 1        |
| P <sub>2</sub> | 5     | 1            | 2        |
| P <sub>3</sub> | 3     | 2            | 2        |
| P <sub>4</sub> | 7     | 3            | 3        |



9 In una situazione dove un processo con priorità 3 e Burst t. di 5, Arrival time 0, inizia l'esecuzione, ma all'istante 2 arriva un processo di priorità 1, quindi con Arrival time 2, Burst time 5, il processo corrente in esecuzione viene fermato e sostituito con il processo di priorità 1, al termine della sua esecuzione non arrivano altri processi con priorità maggiore riprendere quello con priorità 3, dal punto in cui è stato stoppato. (Burst time  $5 - 2 = 3$ ) perché è stato stoppato all'istante 2.

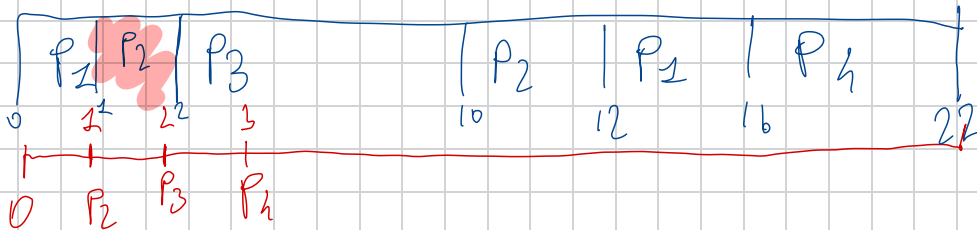
### ★ Priority

| Proc. | Burst | Priority |
|-------|-------|----------|
| P1    | 10    | 3        |
| P2    | 1     | 1        |
| P3    | 2     | 4        |
| P4    | 1     | 2        |



# SJF Priorità e selezione

| Processi       | Burst | Arrival time | Priority |
|----------------|-------|--------------|----------|
| P <sub>1</sub> | 5     | 0            | 2°       |
| P <sub>2</sub> | 3     | 1            | 2°       |
| P <sub>3</sub> | 8     | 2            | 1°       |
| P <sub>4</sub> | 6     | 3            | 3°       |



Coda BT

~~P<sub>1</sub> (4)~~

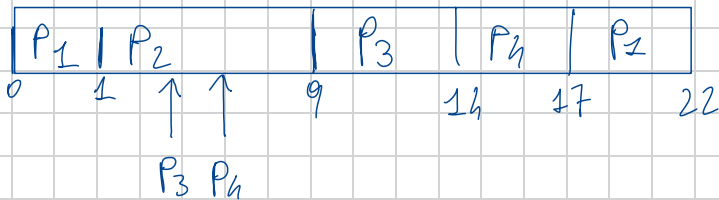
~~P<sub>2</sub> (2)~~

P<sub>4</sub> (6)

## ★ Priority Preemptive

|                | Burst | Arrival | Priority |
|----------------|-------|---------|----------|
| P <sub>1</sub> | 6     | 0       | 3        |
| P <sub>2</sub> | 8     | 1       | 1        |
| P <sub>3</sub> | 5     | 2       | 2        |
| P <sub>4</sub> | 3     | 3       | 2        |

these priority is  
schedule in base  
all arrival time

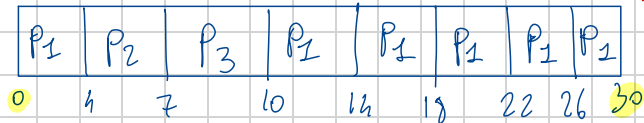


## ★ Round Robin = RR = alg circolare

| Proc.          | BURST |
|----------------|-------|
| P <sub>1</sub> | 24 20 |
| P <sub>2</sub> | 3 ✓   |
| P <sub>3</sub> | 3 ✓   |

quanto = 4 ms

code:



conviene farlo con una parte dove segniamo le code  
processi pronti.

|                | BT             | AT | $q = 3ms$ |
|----------------|----------------|----|-----------|
| P <sub>1</sub> | <del>6</del> 3 | 0  |           |
| P <sub>2</sub> | <del>8</del> 1 | 1  |           |
| P <sub>3</sub> | <del>4</del> 2 | 3  |           |
| P <sub>4</sub> | <del>6</del> 5 | 5  |           |
| P <sub>5</sub> | <del>6</del> 7 | 7  |           |

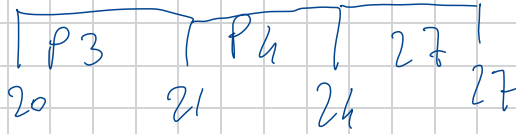
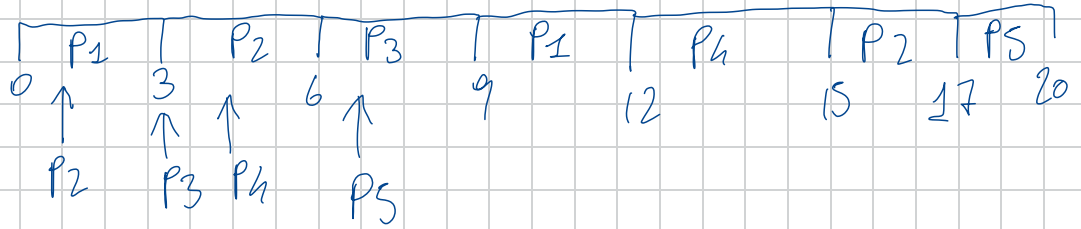
codice

~~P<sub>5</sub>~~

~~P<sub>3</sub>~~

P<sub>4</sub>

P<sub>5</sub>



\* temp attesa = t. completamento - t. di arrivo - Burst time

\* temp. rise. = t. inizio esecuzione - t. arrivo

\* tourn around = t. complet. - t. arrivo ←

\* tourn around = t. attesa + burst time ←

risposta

P<sub>1</sub> (0-0) P<sub>5</sub> (17-7)

P<sub>2</sub> (3-1)

P<sub>3</sub> (6-3)

P<sub>4</sub> (12-5)

t. attesa

P<sub>1</sub> (12-0-6)

P<sub>2</sub> (17-1-5)

P<sub>3</sub> (21-3-4)

P<sub>4</sub> (24-5-6)

P<sub>4</sub> (27-7-6)

## ES. MEMORIA PAGINATA

$$2^{10} = \text{Kb} \quad 2^{20} = \text{Mb}$$

$$2^{30} = \text{Gb}$$

ind logici 2048 pagine con pagine 8Kb mappate su mem. Fisica di 1024 frame

TRACCIA luglio

ES 5

① bit ind logico = 24 Bit

② bit ind fisico = 23 Bit

③ Entry per una tab. PAG ?

④ Quante Entry avrà Pag 2° lv. se il 1° lv. è di 4Kb?

① •  $2048 = 2^{11} \rightarrow 11 \text{ bit per pagine}$

•  $8\text{kb} \rightarrow 8 \cdot 1024 = 8192 \rightarrow 2^{13} \rightarrow 13 \text{ bit per l'offset}$   
offset per pagine.

• Bit indirizzato logico = Bit per pagine (11 bit) + Bit offset (13 bit)  
Ris = 24 Bit

② - 1024 frame =  $2^{10} \rightarrow 10 \text{ bit per identificare un frame}$   
- 13 bit offset (calcolato prima)

• Bit indirizzato Fisico = bit per identificare FRAME (10 bit + OFFSET (13 bit))  
Risultato = 23 bit



NON CI SONO NELLA FRACCIA

(3) calcolo entry in una tab. delle pagine

$$\begin{array}{l} \text{IND. LOGIC} = 24 \text{ bit} \rightarrow 2^{24} \\ \text{DIM. PAG} = 8 \text{ kb} \rightarrow 2^{13} \end{array} \left. \vphantom{\begin{array}{l} \text{IND. LOGIC} = 24 \text{ bit} \rightarrow 2^{24} \\ \text{DIM. PAG} = 8 \text{ kb} \rightarrow 2^{13} \end{array}} \right\} \text{CALCOLI:}$$
$$\frac{2^{24}}{2^{13}} = 2^{11} = 2048$$

- NUMERO ENTRY PER UNA TAB. DELLE PAG. = 2048 entry

numero entry (2048) è lo stesso del numero di PAGINE  
perché c'è una entry per PAGINA

## ④ ALGORITHM BANCHIERE / dead lock

DATI: Allocation, Max, A = 10 istanze, B = 5 ist., C = 7 ist.

da calcolare: vettore Available, matrice Need / Bisogno

• Allocation

|                | A | B | C |
|----------------|---|---|---|
| P <sub>0</sub> | 0 | 1 | 0 |
| P <sub>1</sub> | 2 | 0 | 0 |
| P <sub>2</sub> | 3 | 0 | 2 |
| P <sub>3</sub> | 2 | 1 | 1 |
| P <sub>4</sub> | 0 | 0 | 2 |

• Max

|                | A | B | C |
|----------------|---|---|---|
| P <sub>0</sub> | 7 | 5 | 3 |
| P <sub>1</sub> | 3 | 2 | 2 |
| P <sub>2</sub> | 9 | 0 | 2 |
| P <sub>3</sub> | 2 | 2 | 2 |
| P <sub>4</sub> | 4 | 3 | 3 |

$$A = 10 - (0 + 2 + 3 + 2 + 0) = 3$$

$$B = 5 - (1 + 0 + 0 + 1 + 0) = 3$$

$$C = 7 - (0 + 0 + 2 + 1 + 2) = 2$$

Vettore Available / Bisogno

| A | B | C |
|---|---|---|
| 3 | 3 | 2 |

Risorse attualmente disponibili

• Need =  $\text{Max}(i; j) - \text{Allocation}(i; j)$

|                | A | B | C |
|----------------|---|---|---|
| P <sub>0</sub> | 7 | 4 | 3 |
| P <sub>1</sub> | 1 | 2 | 2 |
| P <sub>2</sub> | 6 | 0 | 0 |
| P <sub>3</sub> | 0 | 1 | 1 |
| P <sub>4</sub> | 4 | 3 | 1 |

• Available

| A  | B | C |                          |
|----|---|---|--------------------------|
| 3  | 3 | 2 | + (2 0 0) P <sub>2</sub> |
| 5  | 3 | 2 | + (2 1 1) P <sub>3</sub> |
| 7  | 4 | 3 | + (0 0 2) P <sub>4</sub> |
| 7  | 4 | 5 | + (3 0 2) P <sub>2</sub> |
| 10 | 4 | 7 | + (0 1 0) P <sub>0</sub> |
| 10 | 5 | 7 |                          |

esiste almeno un Processo

che ha  $\text{Need}[i; j] \leq \text{Available}[i; j]$ ?

P<sub>1</sub> → P<sub>3</sub> → P<sub>4</sub> → P<sub>2</sub> → P<sub>0</sub>

Sistema si trova in uno stato

SICURO / SAFE

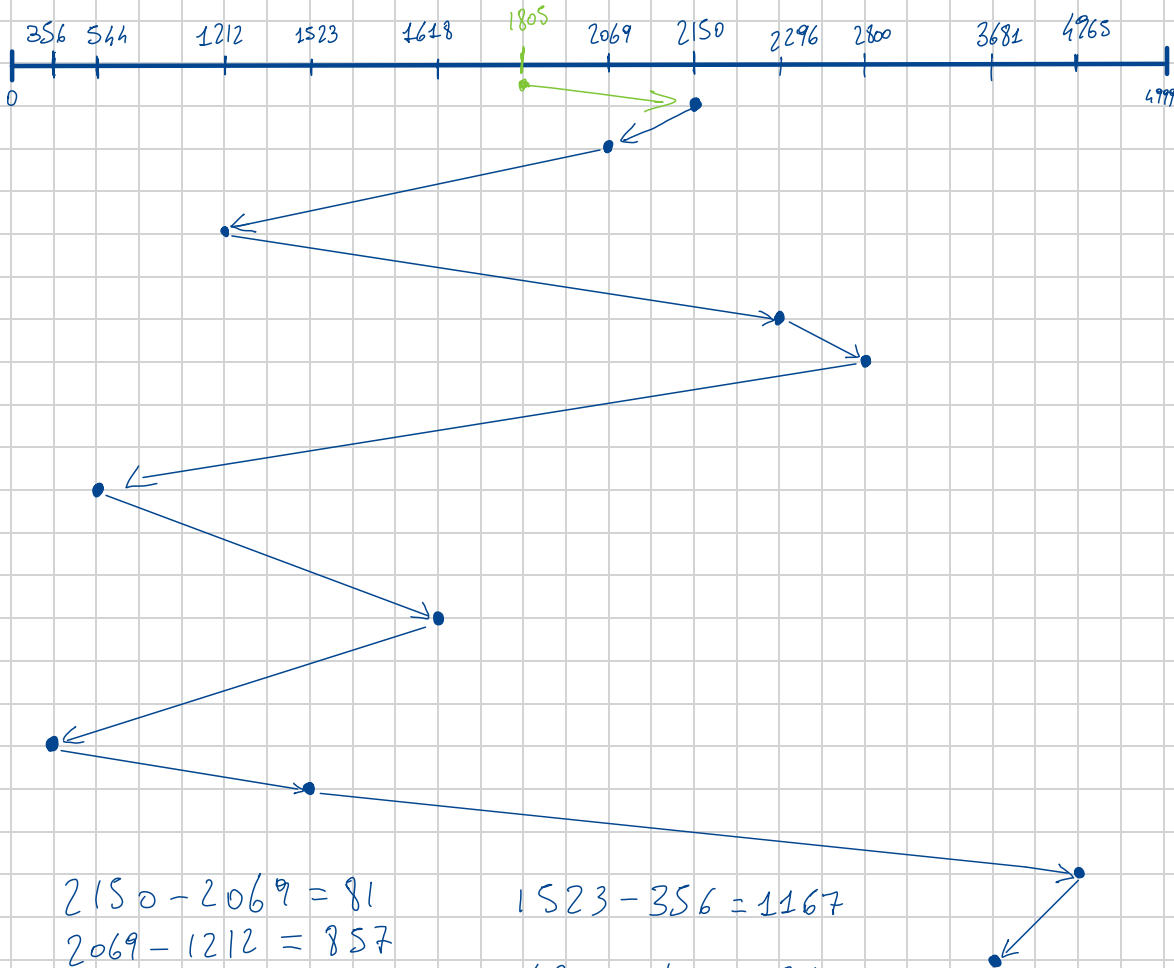
da definire perché è in uno stato sicuro

# HARD DISK

assumendo 5000 cilindri start 2150, richiesta pre(1805)

queue/coda: 2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681

FCFS



$$2150 - 2069 = 81$$

$$2069 - 1212 = 857$$

$$2296 - 1212 = 1084$$

$$2800 - 2296 = 504$$

$$2800 - 544 = 2256$$

$$1618 - 544 = 1074$$

$$1618 - 356 = 1262$$

$$1523 - 356 = 1167$$

$$4965 - 1523 = 3442$$

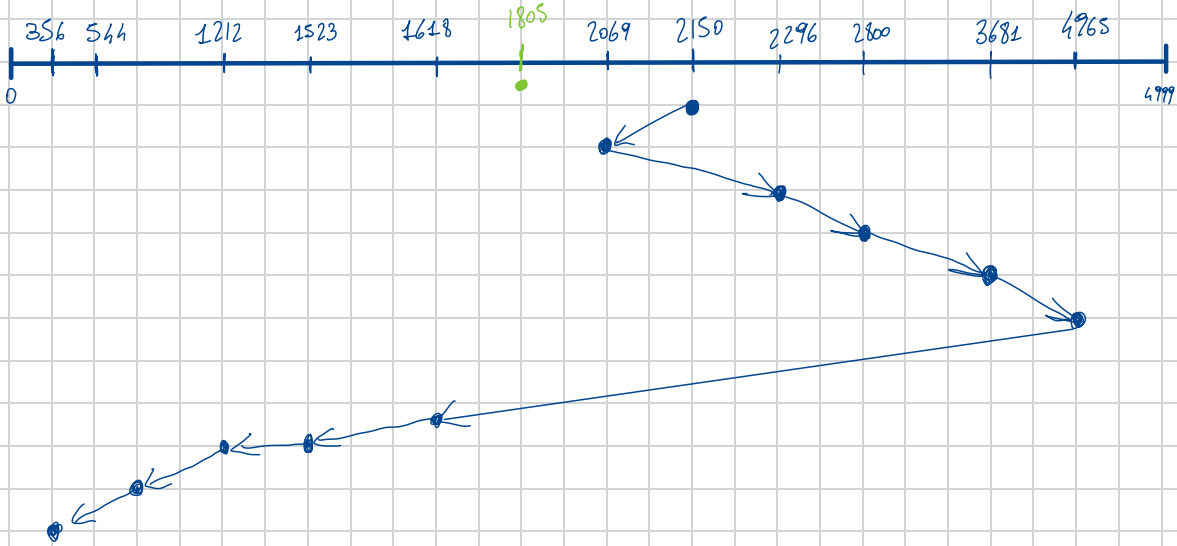
$$4965 - 3681 = 1284$$

---

$$\text{tot} = 13.011 \text{ cilindri}$$

assumendo 5000 cilindri start 2150, richiesta pre(1805)  
queue/code: 2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681

SSTF



Criterio

$$2150 - 2069 = 81 +$$

$$2296 - 2069 = 227 +$$

$$2800 - 2296 = 504 +$$

$$3681 - 2800 = 881 +$$

$$4965 - 3681 = 1284 +$$

$$4965 - 1618 = 3347 +$$

$$1618 - 1523 = 95 +$$

$$1523 - 1212 = 311 +$$

$$1212 - 544 = 668 +$$

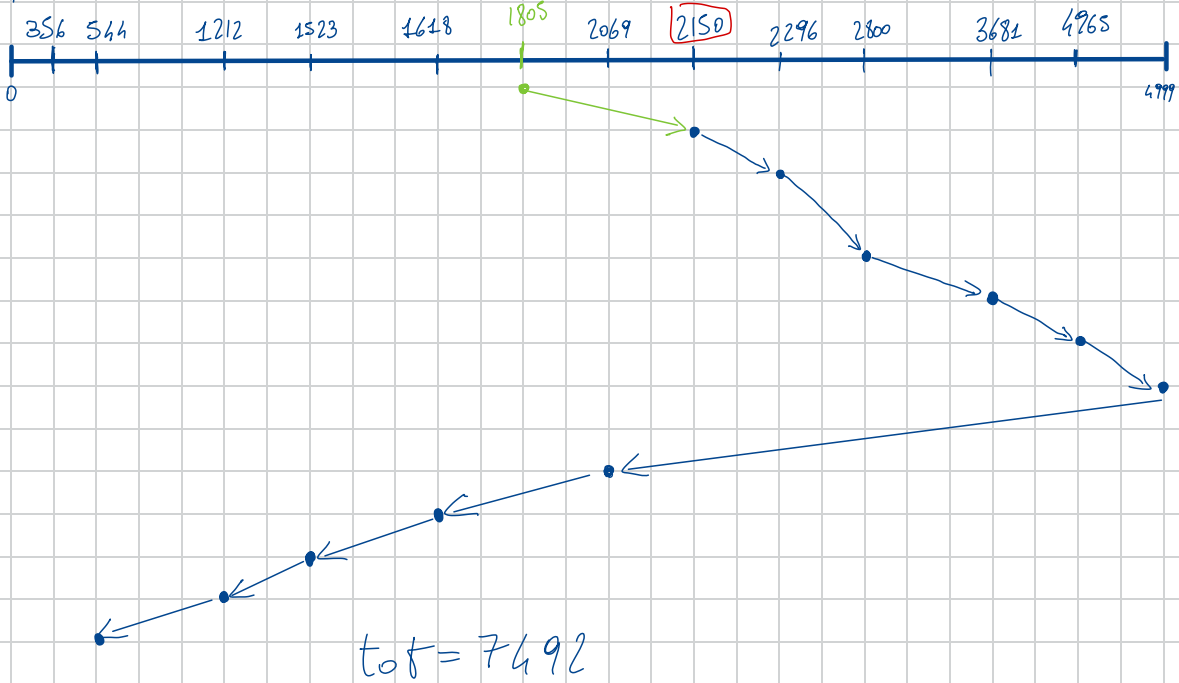
$$544 - 356 = 188 = 7586 \text{ cilindri}$$

Si serve sempre quella più vicina  
al punto in cui si trova la testina,  
fare sempre le sottrazioni perché a volte  
la differenza tra un punto e l'altro  
può essere di poche unità.

## SCAN

assumendo 5000 cilindri start 2150, richiesta pre(1805)

queue/coda: 2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681

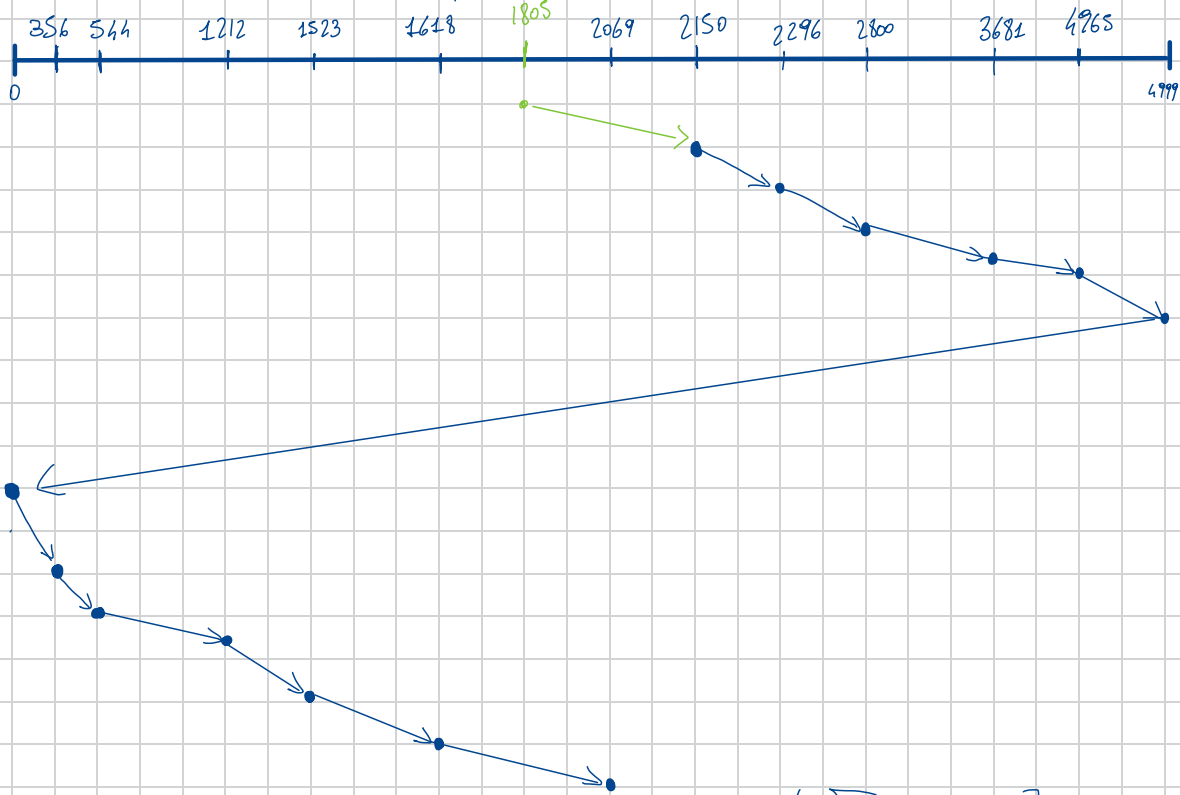


direzione richieste precedente tocca l'estremo che si trova in quella direzione, una volta arrivato all'estremo inverte le rotte e va alle richieste più vicine allo start disponibile per poi iniziare a servire tutte le richieste nell'altra direzione.

## C-SCAN

assumendo 5000 cilindri start 2150, richiesta pre(1805)

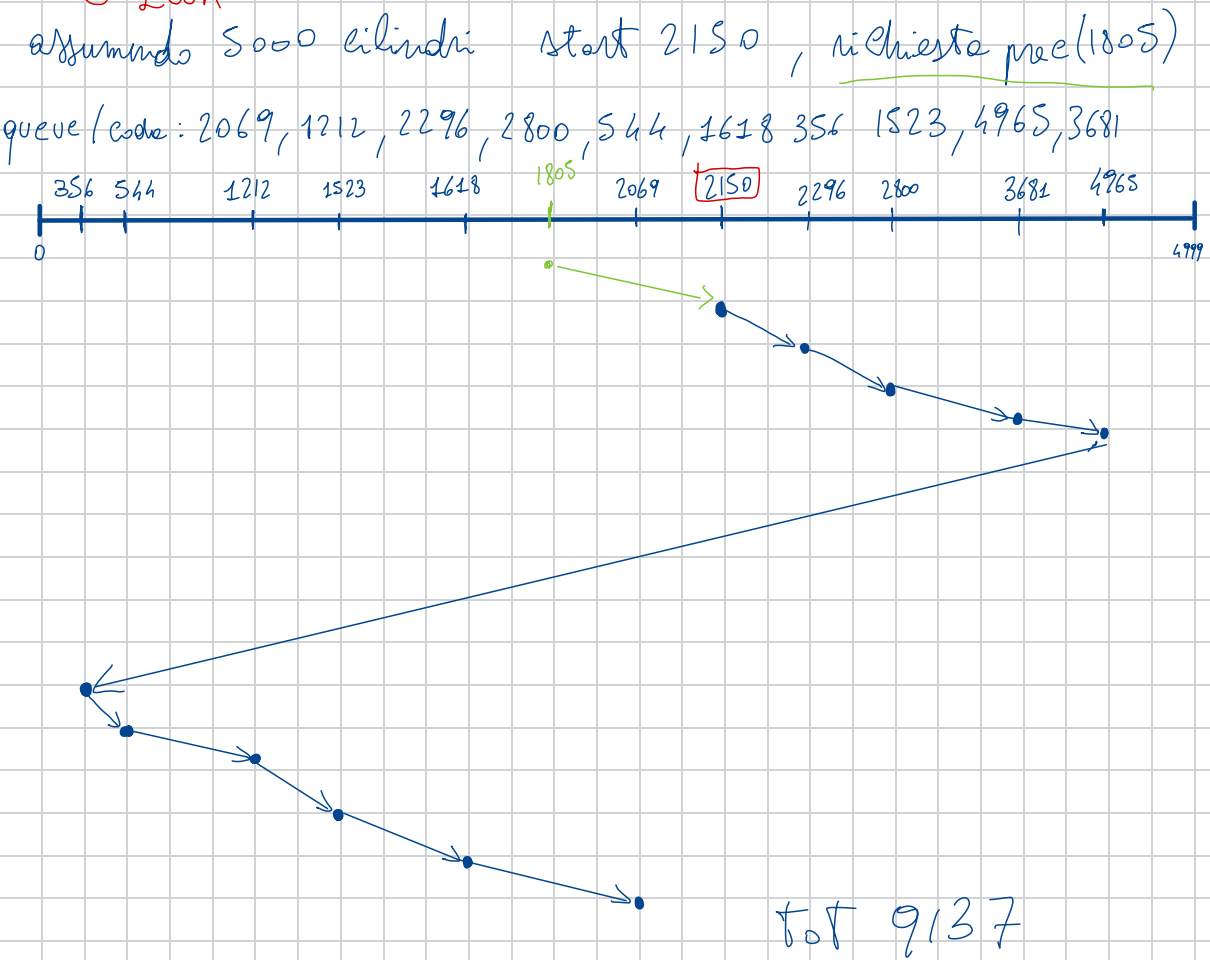
queue/code: 2069, 1212, 2296, 2800, 544, 1618 356 1523, 4965, 3681



$$TOT = 9917$$

direzione richiesta precedente, serve tutte le rich. in quella direzione tocca l'estremo, inverte andando all'estremo opposto da lì inizia servire tutte le richieste fino ad arrivare a quella più vicina allo start, (OVVIAMENTE start servite per prime, quindi già occupate)

## C-Look



serve tutte le richieste in direzione della richiesta precedente arrivata all'ultimo in quella direzione invertita e va alle richieste più vicine all'estremo opposto servendo poi tutte le altre fino ad arrivare alla richiesta più vicina allo start.



PAGE FAULT, ALG: FIFO, LRU, OPT = ALG. OPTIMAL

Minimizzare numero di page fault 3 frame, riferimento:

in memoria: richieste alla memoria da parte dei processi.

- **FIFO** = first in First out

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1,

|   |
|---|
| 7 |
|---|

|   |   |
|---|---|
| 7 | 0 |
|---|---|

|   |   |   |
|---|---|---|
| 7 | 0 | 1 |
|---|---|---|

|   |   |   |
|---|---|---|
| 2 | 0 | 1 |
|---|---|---|

|   |   |   |
|---|---|---|
| 2 | 3 | 4 |
|---|---|---|

|   |   |   |
|---|---|---|
| 2 | 3 | 0 |
|---|---|---|

|   |   |   |
|---|---|---|
| 4 | 2 | 0 |
|---|---|---|

|   |   |   |
|---|---|---|
| 4 | 2 | 3 |
|---|---|---|

|   |   |   |
|---|---|---|
| 0 | 2 | 3 |
|---|---|---|

|   |   |   |
|---|---|---|
| 0 | 1 | 3 |
|---|---|---|

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
|---|---|---|

|   |   |   |
|---|---|---|
| 7 | 1 | 2 |
|---|---|---|

|   |   |   |
|---|---|---|
| 7 | 0 | 2 |
|---|---|---|

|   |   |   |
|---|---|---|
| 7 | 0 | 1 |
|---|---|---|

Lehrer

Result of 2 = 15 page Fault

conte N° Bloccati:

- LRU simile al FIFO

[illegible]

per ordine prestati: posizionare i numeri in base all'ordine di arrivo per evitare equivoci esempio

Resultato = 12 page fault

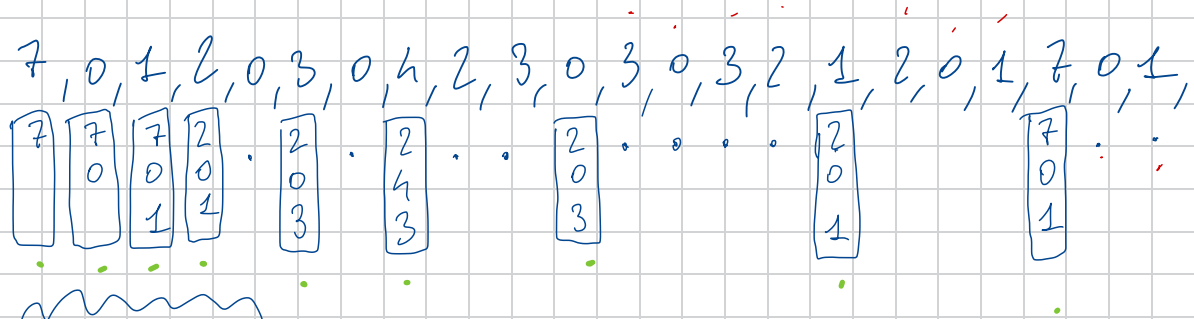
quest'alg. quando incontra un numero presente in uno dei 3 Frame fa un refresh del numero rendendolo il più "giovane" tra i 3 da quell'istante in poi

1° più giovane

2°

3° più anziano

- **OPT:** si suppone di sapere riferimenti futuri, sacrificando il meno ricorrente o quello che non utilizzerò più in futuro o che non utilizzerò per maggior tempo.



Risultato = 9 page Fault.

OPT = non può esistere perché non conosciamo né possiamo prevedere i riferimenti futuri.

## esercizio Hit, miss

accesso in memoria 100ns, Hit 80%, t. accesso = 3ns

Dati impliciti se l'hit è di 80% miss sarà di conseguenza 20%.

- ① tempo accesso in MEMORIA paginate senza TLB (cache specializzata per i tempi accesso in mem)
- ② t. medio accesso in MEMORIA con TLB

①  $100 \times 2 = 200 \text{ ns}$  controllo

②  $\text{miss} * t_{\text{acc. mem. principale}} + t_{\text{acc. TLB}} + \text{Hit rate} * t_{\text{accesso}}$   
 $0,20 * 200 + 3 + 0,80 * 100 = 123 \text{ ns}$

working set

(a) Si descriva brevemente il concetto di *working set*  $WS(t, \Delta)$ , all'istante  $t$  con intervallo  $\Delta$ .

(b) Si consideri la seguente stringa di riferimenti (partendo con  $t = 0$ ):

2 6 5 7 7 7 5 1 6 4  
 ↑ ↑ ↑  
 8 10 11

Cosa è  $WS(10, 8)$ , ossia dopo l'ultimo accesso?

(a) Il working set è un'approssimazione della località del processo, ossia è l'insieme di pagine "attualmente" riferite. In generale  $WS(t, \Delta) = \text{insieme delle pagine riferite negli accessi } [(t - \Delta + 1), t]$ .

(b)  $WS(10, 8) = \{1, 4, 5, 6, 7\}$

si scrive la sequenza senza duplicati

**NB** la  $t$  può partire sia da 0 che da 1, mentre  $\Delta$  sempre da 1 partendo dalla  $t$  che abbiamo trovato

8. Descrivere in maniera sintetica ma esaustiva il concetto di working set  $WS$  all'istante  $t$  con intervallo  $\Delta$   $WS(t, \Delta)$  e la sua funzione.

a) Data la stringa dei riferimenti 5, 4, 1, 6, 2, 2, 7, 10, 7 scrivere il contenuto di  $WS(9, 4)$

9. Spiegare brevemente la problematica del thrashing.

Risultato = 2, 7, 10, 7 leva doppia → 2, 7, 10 R. Finale

#### 4. Nel contesto dei metodi per evitare i deadlock:

a) Dati tre thread T<sub>0</sub>, T<sub>1</sub>, T<sub>2</sub> e tre tipi di risorsa A (5 istanze), B (5 istanze), C (5 istanze) con le seguenti matrici di

Allocazione e Max:

|                | Allocazione |   |   |   | Max |   |   |
|----------------|-------------|---|---|---|-----|---|---|
|                | A           | B | C |   | A   | B | C |
| T <sub>0</sub> | 1           | 1 | 1 |   | 5   | 4 | 4 |
| T <sub>1</sub> | 1           | 0 | 1 |   | 4   | 3 | 3 |
| T <sub>2</sub> | 0           | 2 | 0 | ✓ | 3   | 4 | 3 |

b) calcolare il vettore Disponibile e la matrice Bisogno

c) Verificare se il sistema è in uno stato sicuro (spiegare i passaggi)

d) Può T<sub>0</sub> richiedere (0, 1, 0)? Se viene assegnato cosa succede?

Vett. available  $\Rightarrow$

|   |   |   |
|---|---|---|
| A | B | C |
| 3 | 2 | 3 |

$$A = 5 - (1 + 1 + 0) = 3$$

$$B = 5 - (1 + 0 + 2) = 2$$

$$C = 5 - (1 + 1 + 0) = 3$$

|                | Need |   |   | calcoli         |
|----------------|------|---|---|-----------------|
|                | A    | B | C |                 |
| T <sub>0</sub> | 4    | 3 | 3 | (5 4 4 - 1 1 1) |
| T <sub>1</sub> | 3    | 3 | 2 | (4 3 3 - 1 0 1) |
| T <sub>2</sub> | 3    | 2 | 3 | (3 4 3 - 0 2 0) |

criterio

esiste  $Need \leq available$ ?

$t_2 \rightarrow t_1 \rightarrow T_0$

Stato sicuro  
perché siamo riusciti a liberare le  
risorse di tutti e 3 i processi.

Vett. available

A B C

3 2 3 + 0 2 0

3 4 3 + 1 0 1

4 4 4 + 1 1 1

5 5 5

to può richiedere 0, 1, 0?

4. Nel contesto dei metodi per evitare i deadlock:

a) Dati tre thread T0, T1, T2 e tre tipi di risorsa A (5 istanze), B (5 istanze), C (5 istanze) con le seguenti matrici di

Allocazione e Max:

|    | Allocazione |   |   |  | Max |   |   |
|----|-------------|---|---|--|-----|---|---|
|    | A           | B | C |  | A   | B | C |
| T0 | 1           | 2 | 1 |  | 5   | 4 | 4 |
| T1 | 1           | 0 | 1 |  | 4   | 3 | 3 |
| T2 | 0           | 2 | 0 |  | 3   | 4 | 3 |

b) calcolare il vettore Disponibile e la matrice Bisogno

c) Verificare se il sistema è in uno stato sicuro (spiegare i passaggi)

d) Può T0 richiedere (0, 1, 0)? Se viene assegnato cosa succede?

Available  $\Rightarrow$

| A | B | C |
|---|---|---|
| 3 | 1 | 3 |

$$A = 5 - (1 + 1 + 0) = 3$$

$$B = 5 - (2 + 0 + 2) = 1$$

$$C = 5 - (1 + 1 + 0) = 3$$

Need

calcoli

$$T_0 \quad 4 \quad 2 \quad 3 \quad (5 \ 4 \ 4 - 1 \ 2 \ 1)$$

$$T_1 \quad 3 \quad 3 \quad 2 \quad (4 \ 3 \ 3 - 1 \ 0 \ 1)$$

$$T_2 \quad 3 \quad 2 \quad 3 \quad (3 \ 4 \ 3 - 0 \ 2 \ 0)$$

Stato NON Sic.

5. Si consideri una memoria paginata:  
 Si assuma uno spazio di indirizzi logici di 2048 pagine con pagine da 4 KB mappate su memoria fisica di 512 frame
- Quanti bit occorrono per l'indirizzo logico?
  - Quanti bit occorrono per l'indirizzo fisico?
6. Si consideri un processo che genera la seguente stringa dei riferimenti alle pagine virtuali:  
 4, 2, 3, 4, 4, 2, 2, 4, 3, 2, 2
- Se il processo ha 2 frame gestiti con LRU quanti page fault vengono generati?
  - Quanti page fault vengono generati con l'algoritmo OPT?
7. Assumendo 5000 cilindri e testina su 1000 (con richiesta precedente a 1100)
- Data la coda di richieste 300, 1800, 700, 4800, 1200, 800 come vengono servite le richieste con l'algoritmo LOOK
  - Qual è la distanza in cilindri percorsi della testina con l'algoritmo LOOK?
  - Spiegare brevemente quando è utile lo scheduling del disco

$$2^{10} \text{ Kb} = 1024$$

$$2^{20} \text{ Mb}$$

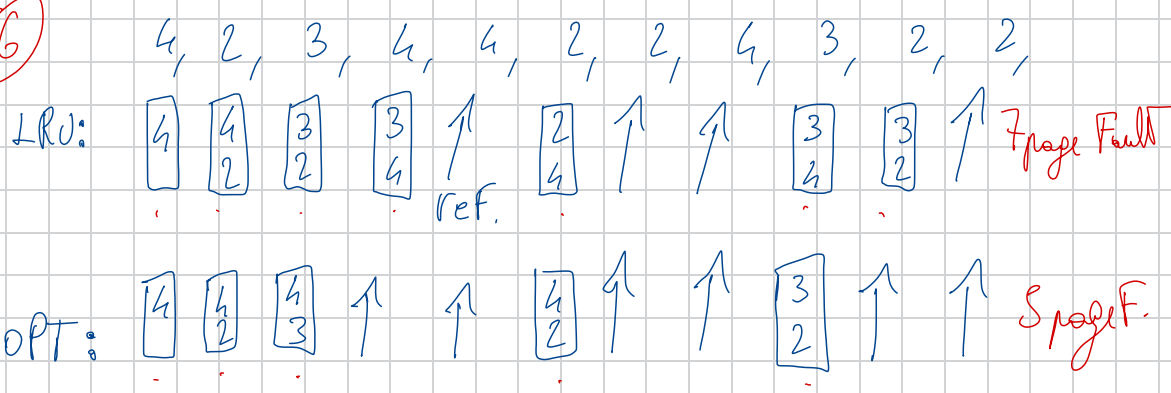
$$2^{30} \text{ GB}$$

5

① ind. logico = 2048  $\rightarrow 2^{11} \rightarrow 11 \text{ bit}$   
 4 Kb = 1024 · 4 = 4096  $\rightarrow 2^{12} \rightarrow 12 \text{ bit offset}$  a pagine  
 ind. logico = 11 + 12 = 23 bit

② indirizzo fisico: 512 Frame  $\rightarrow 2^9 = 9 \text{ bit}$  per indicare un frame  
 offset esodato prima (11 bit)  
 9 + 11 = 20 bit Risultato indirizzo fis.

6



5. Si consideri una memoria paginata:

Si assuma uno spazio di indirizzi logici di 2048 pagine con pagine da 4 KB mappate su memoria fisica di 512 frame

- Quanti bit occorrono per l'indirizzo logico?
- Quanti bit occorrono per l'indirizzo fisico?

6. Si consideri un processo che genera la seguente stringa dei riferimenti alle pagine virtuali:

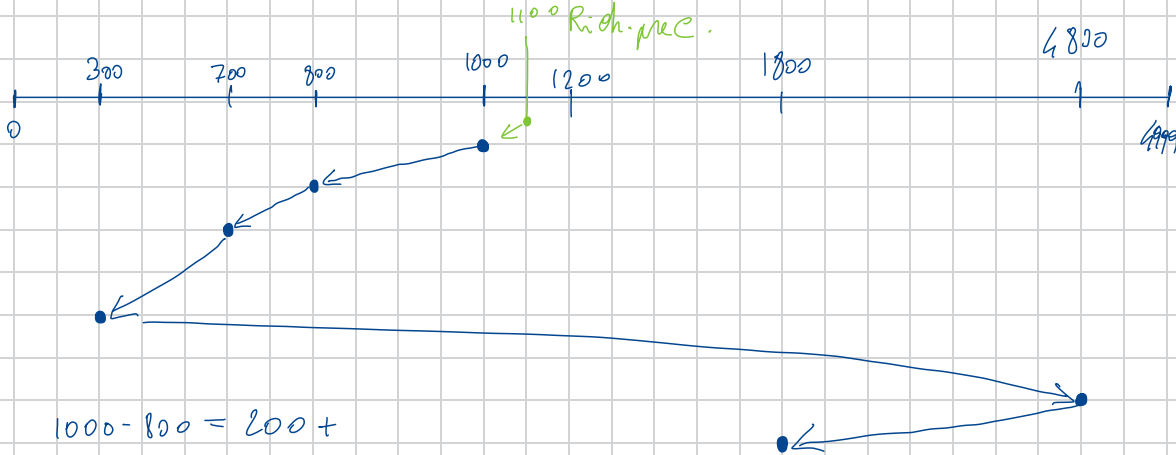
4, 2, 3, 4, 4, 2, 2, 4, 3, 2, 2

- Se il processo ha 2 frame gestiti con LRU quanti page fault vengono generati?
- Quanti page fault vengono generati con l'algoritmo OPT?

7. Assumendo 5000 cilindri e testina su 1000 (con richiesta precedente a 1100)

- Data la coda di richieste 300, 1800, 700, 4800, 1200, 800 come vengono servite le richieste con l'algoritmo LOOK?
- Qual è la distanza in cilindri percorsi della testina con l'algoritmo LOOK?
- Spiegare brevemente quando è utile lo scheduling del disco

LOOK



$$1000 - 800 = 200 +$$

$$800 - 700 = 100 +$$

$$700 - 300 = 400 +$$

$$4800 - 300 = 4500 +$$

$$4800 - 1800 = 3000 =$$

8200 cilindri

## 2. Tenuto conto che:

- il pid del processo padre è 8 e quello del primo figlio è 9 quello del secondo figlio è 10.

Cosa stamperà il seguente programma? Giustificare in estrema sintesi la risposta

```
int var = 20;
```

Figlio risultato Fork = 0

Padre  $n > 0$

che è  $\neq$  da pid

perché pid non è risultato fork.

```
int main()
{
    pid_t pid1, pid2;
    int status = 5;
    pid1 = fork();
    status++; → status = 6
    if (pid1 == 0) {
        var = var + status;
    }
    else {
        pid2 = fork();
        if (pid2 != 0) var = var + pid1 + pid2 + status;
    }
    printf("var = %d \n", var);
    printf("pid = %d \n", getpid());
    return 0;
}
```

nessun risultato qui

$P_1(n > 0)$  pid 8

$F_2(0)$  pid 9

$var(20) + stat(6)$

qui entra solo padre  
ed esegue una fork con pid 2,

quindi si sdoppia in  
un figlio, quindi da  
quella Fork uscirà:

Output

$P_1 \quad var = var + pid1 + pid2 + status;$

$20 + 9 + 10 + 6 = 45$

pid 8

$pid2 = fork()$   $P_1(n > 0)$  pid 8

$F_2(0)$  pid 10

risultato nella variabile

$F_1 \quad var = 26, pid = 9$

$F_2 \quad var = 20, pid = 10$  perché viene creato nell'else  
salta la cond if (pid2 != 0)



```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
```

Pid del sistema operativo  
Padre 10, Figlio 11

OUTPUT

```
Inserisci il valore di a: 3
Inserisci il valore di b: 4
Figlio: Il risultato è: 12
Padre: Il risultato è: 7
```

```
// Variabile globale per memorizzare l'operazione
int operazione;

// Funzione universale per la stampa dei risultati
void stampa_risultato(const char *chi, int risultato) {
    printf("%s: Il risultato è: %d\n", chi, risultato);
}

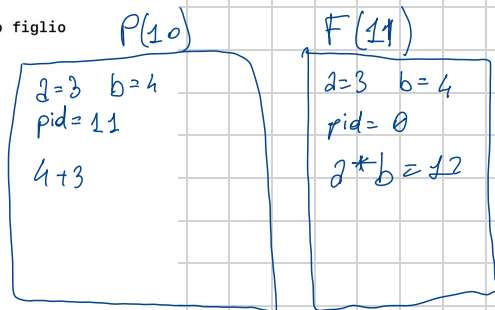
int main() {
    int a, b;           // Variabili per i numeri inseriti dall'utente
    int resultfork;     // Variabile per memorizzare il risultato del processo figlio
    pid_t pid;          // Variabile per memorizzare l'ID del processo

    // Richiesta di input all'utente
    printf("Inserisci il valore di a: ");
    scanf("%d", &a); // ← 3
    printf("Inserisci il valore di b: ");
    scanf("%d", &b); // ← 4

    // Creazione del processo figlio
    pid = fork();

    if (pid < 0) { // Errore nella creazione del processo
        perror("Errore nella fork");
        exit(1);
    } else if (pid == 0) { // Codice eseguito dal processo figlio
        operazione = a * b;
        stampa_risultato("Figlio", operazione); // Utilizzo della funzione di stampa universale
        exit(0); // Termina il processo figlio
    } else { // Codice eseguito dal processo padre
        wait(NULL); // Attende che il processo figlio termini
        operazione = a + b;
        stampa_risultato("Padre", operazione); // Utilizzo della funzione di stampa universale
    }

    return 0;
}
```



## SPIEGAZIONE IMPORTANTE

il programma si sdoppia partendo dalla variabile globale, le locali  $a$  e  $b$ , con i valori inseriti dall'utente

Padre

Figlio

$a=3; b=4$

$a=3; b=4$

$pid=11$

$pid=0$  perché la `Fork()` nel figlio restituisce 0

le `Fork()` nel padre restituisce l'identificativo (`pid`) del figlio,

3. Dato il frammento di programma che segue:

- Quanti processi e quanti thread genera il frammento di programma seguente?
- Evidenziare, se ci sono, le corse critiche.
- Cosa stamperà il programma?

Giustificare in estrema sintesi le risposte

```
#define NUM_THREADS 3
```

```
int var1 = 10, var2 = 0;
```

```
void *func1(void* param)
```

```
{
    var1 -= 1;
```

```
    return 0;
}
```

```
void *func2(void* param)
```

```
{
    var2 += 1;
```

```
    return 0;
}
```

```
int main()
```

```
{
    pthread_t threads[NUM_THREADS];
```

```
    pthread_create(&threads[0], NULL, func1, NULL);
```

```
    pthread_create(&threads[1], NULL, func2, NULL);
```

```
    pthread_join(threads[0], NULL);
```

```
    pthread_join(threads[1], NULL);
```

```
    fork();
```

```
    pthread_create(&threads[1], NULL, func2, NULL);
```

```
    pthread_create(&threads[2], NULL, func1, NULL);
```

P

var1 = 8  
 var2 = 2

F

var1 = 8  
 var2 = 2

```
pthread_join(threads[1], NULL);
```

```
pthread_join(threads[2], NULL);
```

```
printf("%d %d\n", var1, var2);
```

```
return 0;
```

```
}
```

8, 2

8, 2

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
```

```
int main()
{
    pid_t pid;
    int status, var = 5;
    printf("sono il processo
    padre. \n");
```

pid = fork(); // non è il pid effettivo, dice se è padre o figlio

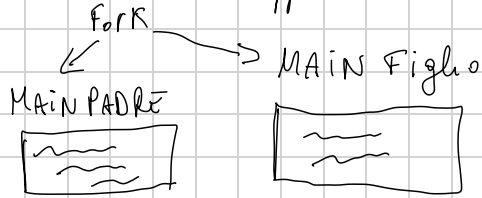
```
if(pid > 0){
    var++; // var=6
    printf("var = %d \n", var);
```

wait(&status); → istruzione di attesa PADRE FINCHÉ IL FIGLIO NON TERMINA  
} attende FINE Figlio, IL PADRE

```
printf("var = %d \n", var); // var=5 nel figlio
printf("pid = %d \n",
getpid()); // Restituisce il pid del processo corrente

return 0;
}
```

quando arriviamo al fork il  
main si sdoppia:



Spiegazione:

1. Funzione fork():

- Quando viene chiamata fork(), crea un nuovo processo (processo figlio) che è un duplicato del processo padre. La funzione fork() restituisce:
- > 0 nel processo padre (il valore restituito è il PID del figlio).
- 0 nel processo figlio.
- < 0 se il fork fallisce.

2. Esecuzione del processo padre:

- Inizialmente, var è impostato su 5. ✓
- Il processo padre stampa "sono il processo padre". ✓
- Il processo padre incrementa var di 1 (quindi var diventa 6) e stampa il valore di var. ✓
- Il processo padre attende quindi che il processo figlio venga completato con wait(&status). ALLA FINE del completamento
- Dopo che il processo figlio è stato completato, il processo padre stampa il valore di var e il suo PID.

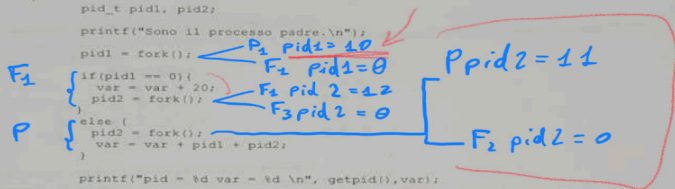
3. Esecuzione del processo figlio:

- Il processo figlio inizia con una copia della memoria del processo padre, quindi var è anche 5 nel figlio.
- Il processo figlio non esegue il blocco all'interno della condizione if(pid > 0) perché pid == 0 per il figlio.
- Il processo figlio stampa direttamente il valore di var (che è 5) e il suo PID.

eseguire sul pc per vedere come si comporta.

1D: P9 F<sub>1</sub> 10 F<sub>2</sub> 11 F<sub>3</sub> 12

```
int var = 50;
int main()
{
    pid_t pidi, pid2;
    printf("Sono il processo padre.\n");
    pidi = fork();
    if (pid1 == 0) {
        var = var + 20;
        pid2 = fork();
    }
    else {
        pid2 = fork();
        var = var + pidi + pid2;
    }
    printf("pid = %d var = %d \n", getpid(), var);
    return 0;
}
```



OUTPUT

| getpid         | pid | var |
|----------------|-----|-----|
| P              | 9   | 74  |
| F <sub>1</sub> | 10  | 70  |
| F <sub>2</sub> | 11  | 60  |
| F <sub>3</sub> | 12  | 70  |

3. Dato il frammento di programma che segue:
- Quanti processi e quanti thread genera il frammento di programma seguente?
  - Evidenziare, se ci sono, le corse critiche.
  - Cosa stamperà il programma?

Giustificare in estrema sintesi le risposte

```
#define NUM_THREADS 3
int var1 = 10, var2 = 0;
void *func1(void* param)
{
    var1 -= 5;
    return 0;
}

void *func2(void* param)
{
    var2 = 5 + var1 + var2;
    return 0;
}
```

2) 2 Proc.  
6 threads

```
int main()
{
    pthread_t threads[NUM_THREADS];
    fork();
    pthread_create(&threads[0], NULL, func1, NULL);
    pthread_create(&threads[1], NULL, func2, NULL);
    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);

    pthread_create(&threads[2], NULL, func2, NULL);
    printf("%d %d\n", var1, var2);
}
```

2T  
+  
1t

→ var1 = 10 - 5  
var2 = 5 + 5 + 0

var2 = 5 + 5 + 10

P → var1 [5] var2 [20]  
F → [5] [20]

# OUTPUT

qui era ancora in esecuzione il processo padre prima di passare la palla al figlio!

```
sono il processo padre.  
var = 6  
var = 5  
pid = [Child's PID]  
var = 6  
pid = [Parent's PID]
```

Stampa prima il figlio perché padre è in wait

- "sono il processo padre." viene stampato dal processo padre.
- Dopo il fork, il processo padre incrementa var a 6 e stampa "var = 6".
- Il processo figlio stampa "var = 5" poiché inizia con var = 5 (indipendentemente dall'incremento del padre).
- Il processo figlio stampa il proprio PID.
- Dopo il completamento del processo figlio, il processo padre riprende, stampa "var = 6" (la sua var era già stata incrementata a 6) e il suo PID.

I PID esatti saranno specifici del sistema.

## - SECONDO CODICE

Il codice C fornito usa il multithreading per incrementare una variabile condivisa. Il risultato di questo codice dipende dal fatto che i thread siano eseguiti in modo thread-safe. Analizziamo il codice passo dopo passo:

Analisi del codice:

```
#define NUM_THREADS 4  
int shared = 0; // variabile condivisa GLOBAL
```

```
void * func (void* param){  
    for(int i=0; i<100; ++i){  
        shared+=1;  
    }  
    return 0;  
}
```

```
int main()  
{  
    pthread_t threads[NUM_THREADS];
```

```
    for (int i = 0; i <  
        NUM_THREADS; ++i){  
        pthread_create(&threads[i],  
            NULL, func, NULL);  
    }
```

```
    for (int i = 0; i <  
        NUM_THREADS; ++i){  
        pthread_join(threads[i],  
            NULL);  
    }
```

```
    printf("Shared = %d \n",  
        shared);
```

```
    return 0;  
}
```

Spiegazione:

1. Variabile condivisa:

- La variabile condivisa è una variabile globale, inizialmente impostata su 0. È condivisa tra tutti i thread.

2. Funzione func():

- Ogni thread esegue la funzione func(), che incrementa la variabile condivisa di 1 in un ciclo di 100 iterazioni.
- Se questa funzione viene eseguita senza meccanismi di sincronizzazione, gli incrementi potrebbero interferire tra loro, portando a una condizione di competizione.

3. Creazione e unione di thread:

- NUM\_THREADS è definito come 4, quindi vengono creati 4 thread.
- Ogni thread incrementa la variabile condivisa 100 volte.
- Idealmente, se tutti i thread incrementano correttamente, shared dovrebbe essere  $4 * 100 = 400$  dopo che tutti i thread hanno terminato.

4. Condizione di competizione:

- La variabile condivisa viene incrementata senza alcuna sincronizzazione (ad esempio, mutex) per garantire l'atomicità.
- Senza sincronizzazione, i thread potrebbero leggere, incrementare e riscrivere su shared in modo non atomico, con conseguente perdita di aggiornamenti.
- Ciò significa che il valore finale di shared può essere inferiore a 400, a seconda della tempistica dei cambi di contesto tra i thread.

Possibili output:

- Caso migliore (nessuna condizione di gara):
- Se, per caso, i thread non interferiscono tra loro (il che è improbabile), l'output potrebbe essere Shared = 400.
- Caso più probabile (condizione di gara presente):
- A causa delle condizioni di gara, è probabile che l'output effettivo sia inferiore a 400. Il valore esatto sarebbe non deterministico e varierebbe tra le esecuzioni. Potrebbe essere qualsiasi cosa da vicino a 400 fino a un valore molto più basso, a seconda dell'entità della condizione di gara.

Conclusione:

È probabile che l'output di questo codice vari tra le diverse esecuzioni e potrebbe apparire simile a questo:

```
Shared = <value less than or  
equal to 400>
```



### 3. Dato il frammento di programma che segue:

- Quanti processi e quanti thread genera il frammento di programma seguente?
- Evidenziare, se ci sono, le corse critiche. Come si possono evitare?
- Cosa stamperà il programma?

Giustificare in estrema sintesi le risposte

```
#define NUM_THREADS 3
int var1 = 30, var2 = 3;
```

```
void *func1(void* param)
{
    var2 = var2 + 10;
    return 0;
}
```

```
void *func2(void* param)
{
    var1 = var1 + var2 - 20;
    return 0;
}
```

```
int main()
{
    pthread_t threads[NUM_THREADS];

    fork();
    for (int i = 0; i < NUM_THREADS; ++i) {
        pthread_create(&threads[i], NULL, func1, NULL);
    }
    for (int i = 0; i < NUM_THREADS; ++i) {
        pthread_join(threads[i], NULL);
    }
    for (int i = 0; i < NUM_THREADS; ++i) {
        pthread_create(&threads[i], NULL, func2, NULL);
    }
}
```



P  
3t  
var2 = 39

F  
3t  
var2 = 39

Func1 =  $\begin{matrix} 13 \\ 23 \\ 33 \end{matrix}$

```
for (int i = 0; i < NUM_THREADS; ++i) {
    pthread_join(threads[i], NULL);
}

printf("%d\n", var1);

return 0;
}
```

|           |           |
|-----------|-----------|
| P         | F         |
| var2 = 33 | var2 = 33 |
| var1 = 43 | var1 = 43 |
| 56        | 56        |
| 69        | 69        |

OUTPUT

var1 = 69

N.B.: il lavoro dei threads è accumulativo nelle variabili globali

Nome e cognome

Matricola

La durata della prova è di 2 ore. Scrivere in stampatello maiuscolo. E' obbligatorio restituire il testo e tutti i fogli forniti, anche se non si consegna il compito.

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella, e considerato che l'ordine di arrivo dei processi è P1, P2, P3, P4, P5.

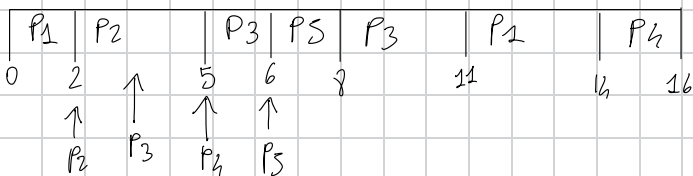
| Processo | Burst Time | Priorità | Tempo di arrivo |
|----------|------------|----------|-----------------|
| P1       | 5          | 3        | 0               |
| P2       | 3          | 1        | 2               |
| P3       | 4          | 2        | 4               |
| P4       | 2          | 4        | 5               |
| P5       | 2          | 1        | 6               |

a) Disegnare 2 diagrammi di Gantt che illustrino l'esecuzione di questi processi usando gli algoritmi di scheduling Priorità con prelazione e RR (quanto = 3 ms). Indicare l'istante di tempo di ciascuna commutazione fra un processo e l'altro.

b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo d'attesa e come l'avete applicato per calcolare i risultati esposti)

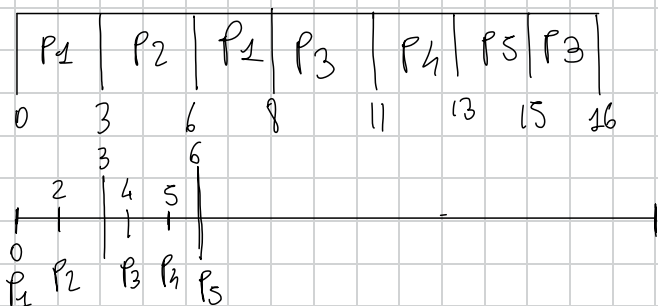
Priority preemptive

tempo attesa:  $t_{\text{complet}} - \text{arr.} - t_{\text{BT}}$



$$\begin{aligned}
 P_1 (11 - 0 - 5) &= 6 \\
 P_2 (3 - 2 - 3) &= 0 \\
 P_3 (8 - 4 - 4) &= 0 \\
 P_4 (16 - 5 - 2) &= 9 \\
 P_5 (6 - 6 - 2) &= 0
 \end{aligned}$$

RR  $q = 3 \text{ ms}$



t. attesa

$$P_1 (11 - 0 - 5) = 6$$

$$P_2 (6 - 2 - 3) = 1$$

$$P_3 (16 - 4 - 4) = 8$$

$$P_4 (13 - 5 - 2) = 6$$

$$P_5 (15 - 6 - 2) = 7$$

Coda

~~P1~~ ✓  
~~P2~~  
~~P3~~ (2)  
 P3 ✓  
~~P4~~  
 P5  
 P3 (1)



$$\text{tourn around (Priority pre.)} = t_{\text{comp}} - t_{\text{arrivo}}$$

$$P_1(14-0)=14 \quad P_2(5-2)=3 \quad P_3(11-4)=7 \quad P_4(16-5)=9$$

$$P_5(8-6)=2$$

$$\star \text{ temp attesa} = t_{\text{completamento}} - t_{\text{di arrivo}} - \text{Burst time}$$

$$\star \text{ temp rise} = t_{\text{inizio esecuzione}} - t_{\text{arrivo}}$$

$$\star \text{ tourn around} = t_{\text{complet.}} - t_{\text{arrivo}}$$

$$\star \text{ tourn around} = t_{\text{attesa}} + \text{burst time}$$

$$\text{tourn around (Round Robin)} = P_1(11-0)=11 \quad P_2(6-2)=4$$

$$P_3(16-4)=12 \quad P_4(13-5)=8 \quad P_5(15-6)=9$$

#### 4. Nel contesto dei metodi per evitare i deadlock:

- a) Dati tre thread T0, T1, T2 e tre tipi di risorsa A (4 istanze), B (4 istanze), C (4 istanze) con le seguenti matrici di Allocazione e Max:

|    | Allocazione |   |   | Max |   |   |
|----|-------------|---|---|-----|---|---|
|    | A           | B | C | A   | B | C |
| T0 | 0           | 2 | 1 | 2   | 3 | 3 |
| T1 | 2           | 1 | 2 | 3   | 2 | 2 |
| T2 | 1           | 0 | 0 | 2   | 2 | 2 |

- b) Calcolare il vettore Disponibile e la matrice Bisogno  
 c) Verificare se il sistema è in uno stato sicuro (spiegare i passaggi)  
 d) Può T2 richiedere subito (1, 0, 0)? Se viene assegnato cosa succede?

$$A = 4 - (0 + 2 + 1) = 1 \quad B = 4 - (2 + 1 + 0) = 1 \quad C = 4 - (1 + 2 + 0) = 1$$

$$\text{Available} = A \ B \ C$$

$$\begin{array}{r} 1 \ 1 \ 1 \\ \hline 3 \ 2 \ 3 \\ \hline 3 \ 4 \ 4 \\ \hline 4 \ 4 \ 4 \end{array} \begin{array}{l} t_1 \\ t_0 \\ t_2 \end{array}$$

$$\text{Need (Max - Allocation)}$$

|    | A | B | C |
|----|---|---|---|
| t0 | 2 | 1 | 2 |
| t1 | 1 | 1 | 0 |
| t2 | 1 | 2 | 2 |

$$\text{Need} \leq \text{Available?}$$

$$t_1 \rightarrow t_0 \rightarrow t_2$$

d)  $t_2(1, 0, 0)$

Allocation

Available

|       | A | B | C |
|-------|---|---|---|
| $t_0$ | 0 | 2 | 1 |

$$A: 4 - (0 + 2 + 2) = 0$$

|       |   |   |   |
|-------|---|---|---|
| $t_1$ | 2 | 1 | 2 |
|-------|---|---|---|

$$B: 4 - (2 + 1 + 0) = 1$$

|       |   |   |   |
|-------|---|---|---|
| $t_2$ | 2 | 0 | 0 |
|-------|---|---|---|

$$C: 4 - (1 + 2 + 0) = 1$$

abbiamo  
aggiunto

Need (Max - Alloc)

|       | A | B | C |
|-------|---|---|---|
| $t_0$ | 2 | 1 | 2 |

|       |   |   |   |
|-------|---|---|---|
| $t_1$ | 1 | 1 | 0 |
|-------|---|---|---|

|       |   |   |   |
|-------|---|---|---|
| $t_2$ | 0 | 2 | 2 |
|-------|---|---|---|

| A | B | C |
|---|---|---|
| 0 | 1 | 1 |

Need  $\leq$  Available?

No, il sistema si trova  
in uno stato non sicuro  
perché non riusciamo a creare  
una sequenza di processi

5. Si consideri una memoria paginata:

Si assumano uno spazio di indirizzi logici di 4096 pagine con pagine di 2 KB mappate su memoria fisica da 512 frame.

- a) Quante sono i bit dell'indirizzo logico?
- b) Quante sono i bit dell'indirizzo fisico?
- c) Quanti entry per una tabella delle pagine invertita?

6. Si consideri un processo che genera la seguente stringa dei riferimenti alle pagine virtuali:

5, 1, 3, 4, 1, 2, 1, 4

- a) Se il processo ha 3 frame gestiti con LRU quanti page fault vengono generati?
- b) Quanti page fault vengono generati con l'algoritmo OPT?

DATI:

Spazio ind. logici = 4096  $\rightarrow 2^{12} \rightarrow 12 \text{ bit per singola pagina}$

pagine di 2 Kb  $\rightarrow 2 \cdot 1024 = 2048 \rightarrow 2^{11} \rightarrow 11 \text{ bit offset}$

512 Frame  $\rightarrow 2^9 \rightarrow 9 \text{ bit per indicare un Frame}$

a) quanti bit per indirizzo logico?

12 bit per singola pagina + 11 bit offset = 23 bit

b) quanti bit ind. Fisico?

9 bit per indicare un Frame + 11 offset = 20 bit

★ c) quante entry per una tabella di pagine invertite?

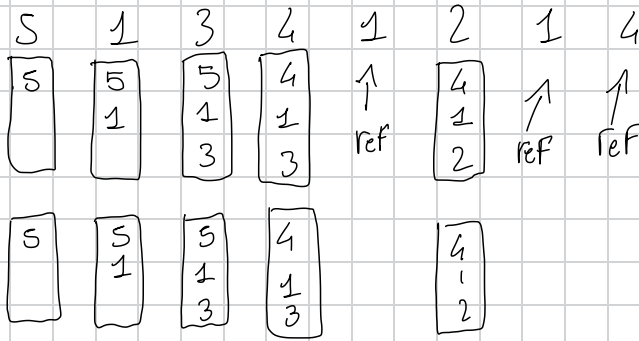
(trovata trovata su Univ. di Roma N frame poi e N entry)

il risultato dovrebbe essere 512 entry, una entry per ogni Frame!

6) Dato  
3 Frame

• LRU  
SpagetF

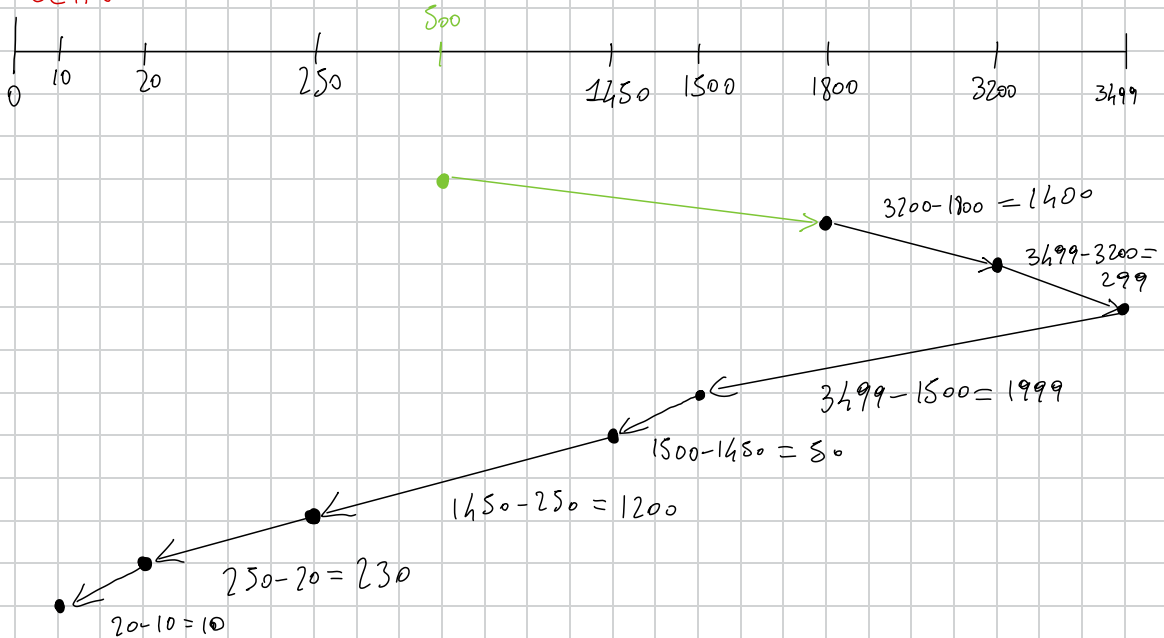
• OPT  
SpagetF



7. Assumendo 3500 cilindri e testina su 1800 (con richiesta precedente a 500)

- a) Data la coda di richieste 10, 1500, 1450, 3200, 20, 250 come vengono servite le richieste con SCAN  
b) Qual è la distanza in cilindri percorsi della testina con l'algoritmo SCAN?

SCAN



tot = 5188 cilindri

8. Spiegare brevemente vantaggi e svantaggi dei metodi di allocazione dei file concatenata e indicizzata.
9. Cos'è un read-write lock? Discutere brevemente il problema dei lettori e scrittori (illustrandone una soluzione).
10. Nell'ambito dello scheduling dei processi spiegare brevemente il problema dell'inversione di priorità.

## 8) Allocazione concatenata dei file

Vantaggi: • implementazione facile perché ogni blocco ha il puntatore al blocco successivo

- Non ha frammentazione esterna: Blocchi contigui su disco, facile trovare spazio libero.
- Crescita dinamica: i file possono crescere o diminuire senza dover spostare dati, perché il blocco può essere aggiunto in qualsiasi posizione.

Svantaggi: • per accedere a un blocco specifico bisogna partire dall'inizio della catena.

- Se un puntatore si corrompe l'accesso ai

File restanti viene perso.

- ogni blocco ha un puntatore che occupa spazio aggiuntivo.

## Allocazione indicizzata dei file

Vantaggi: • indice fornisce accesso diretto a qualsiasi blocco nel File, migliorando Tempi di accesso.

- Non è necessario che i blocchi siano contigui, tabella indice tiene traccia dei blocchi
- i file possono crescere facilmente aggiungendo nuovi blocchi all'indice.

Svantaggi: • tabella degli indici può richiedere molto spazio, soprattutto per File grandi

- Se la Tab. degli indici è di dimensioni fisse, può LIMITARE la dimensione massima del file.

- L'implementazione è più complessa rispetto al metodo

concatenato, in particolare per la gestione di file molto grandi.

9) un read-write lock è un meccanismo di sincronizzazione utilizzato da processi concorrenti per gestire l'accesso ad una risorsa condivisa. Consente a più threads di accedere alla risorsa in modalità lettura CONTEMPORANEAMENTE (lettura non modifica lo stato della risorsa), ma limita l'accesso in SCRITTURA solo a un thread per volta.

Problemi lettori / scrittori

problema che riguarda la sincronizzazione dell'accesso a una risorsa condivisa

Starvation lettori: se diamo priorità agli scrittori, i lettori potrebbero rimanere bloccati a lungo se ci sono scrittori che continuano ad entrare.

Starvation scrittori: dando priorità ai lettori uno

scrittore potrebbe NON riuscire mai a ottenere l'accesso se lettori continuano ad accedere alla risorsa

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella, e considerato che l'ordine di arrivo dei processi è P1, P2, P3, P4, P5.

| Processo | Burst Time | Priorità | Tempo di arrivo |
|----------|------------|----------|-----------------|
| P1       | 5          | 2        | 0               |
| P2       | 4          | 3        | 2               |
| P3       | 4          | 1        | 4               |
| P4       | 5          | 3        | 6               |
| P5       | 4          | 2        | 8               |

a) Disegnare 2 diagrammi di Gantt che illustrino l'esecuzione di questi processi usando gli algoritmi di priorità con prelazione e SJF. Indicare l'istante di tempo di ciascuna commutazione fra un processo e l'altro

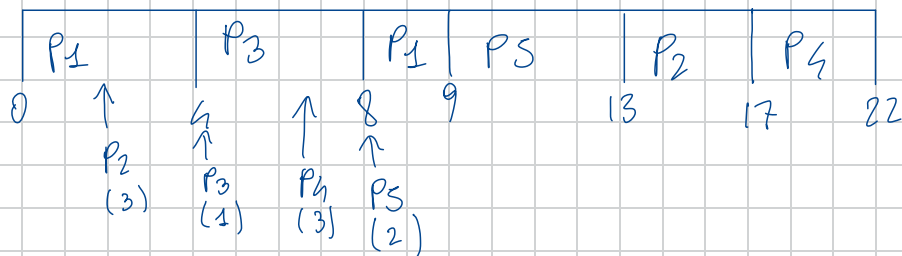
b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo d'attesa e come l'avete applicato per calcolare i risultati esposti)

|    | Priorità (prel) | SJF         |
|----|-----------------|-------------|
| P1 | $(9-0-5)=4$     | $(0-0)=0$   |
| P2 | $(11-2-4)=5$    | $(5-2)=3$   |
| P3 | $(8-4-4)=0$     | $(9-4)=5$   |
| P4 | $(22-6-5)=11$   | $(17-6)=11$ |
| P5 | $(13-8-4)=1$    | $(13-8)=5$  |

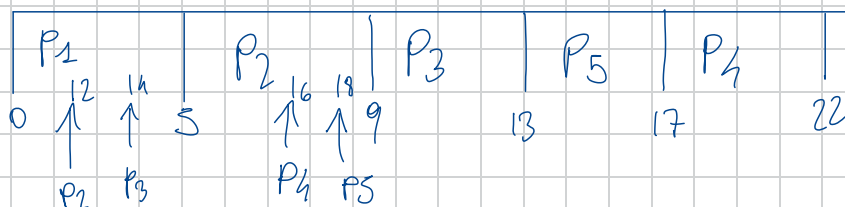
c) Calcolare il tempo di risposta per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo di risposta e come l'avete applicato per calcolare i risultati esposti)

TRACCIA gennaio

Priority preemptive



SJF





$$t. attesa = t. comp - t. arrivo - BurstT.$$

$$t. risposta = t. inizio\ eseguz - t. arrivo$$

Priority preemptive

$$T. attesa = p_1 (9 - 0 - 5) = 4$$

$$p_2 (17 - 2 - 4) = 11$$

$$p_3 (8 - 4 - 4) = 0$$

$$p_4 (22 - 6 - 5) = 11$$

$$p_5 (13 - 8 - 4) = 1$$

$$t. risposta = p_1 (0 - 0) = 0$$

$$p_2 (13 - 2) = 11$$

$$p_3 (4 - 4) = 0$$

$$p_4 (17 - 6) = 11$$

$$p_5 (9 - 8) = 1$$

SJF

$$t.attesa = P_1 (5 - 0 - 5) = 0$$

$$P_2 (9 - 2 - 4) = 3$$

$$P_3 (13 - 4 - 4) = 5$$

$$P_4 (17 - 6 - 5) = 6$$

$$P_5 (22 - 8 - 4) = 10$$

$$t.risposta = P_1 (0 - 0) = 0$$

$$P_2 (5 - 2) = 3$$

$$P_3 (9 - 4) = 5$$

$$P_4 (17 - 6) = 11$$

$$P_5 (13 - 8) = 5$$

4. Nel contesto dei metodi per evitare i deadlock:

- a) Dati tre thread T<sub>0</sub>, T<sub>1</sub>, T<sub>2</sub> e tre tipi di risorsa A (7 istanze), B (4 istanze), C (3 istanze) con le seguenti matrici di Allocazione e Max:

|                | Allocazione |   |   | Max |   |   |
|----------------|-------------|---|---|-----|---|---|
|                | A           | B | C | A   | B | C |
| T <sub>0</sub> | 2           | 2 | 2 | 4   | 4 | 3 |
| T <sub>1</sub> | 1           | 1 | 1 | 2   | 2 | 1 |
| T <sub>2</sub> | 3           | 0 | 0 | 6   | 3 | 3 |

- b) calcolare il vettore Disponibile e la matrice Bisogno  
c) Verificare se il sistema è in uno stato sicuro (spiegare i passaggi)  
d) Può T<sub>1</sub> richiedere (1, 0, 0)? Se viene assegnato cosa succede?

5. Si consideri una memoria paginata, dato un processo con una tabella delle pagine in memoria

- a) Se ogni accesso in memoria richiede 40 ns qual è il tempo per accedere ad un indirizzo della memoria paginata?  
b) Aggiungendo una TLB con hit ratio del 80% e stimando un tempo di accesso di 2 ns qual è il EAT (Effective Access Time)?  
c) Descrivere succintamente cos'è una TLB e qual è la sua funzione

$$\star \text{ temp. attesa} = \underline{t. \text{ completamento}} - \underline{t. \text{ di arrivo}} - \underline{\text{Burst time}}$$

$$\star \text{ temp. risp.} = t. \text{ inizio esecuzione} - t. \text{ arrivo}$$

$$\star \text{ tourn around} = t. \text{ complet.} - t. \text{ arrivo}$$

$$\star \text{ tourn around} = \underline{t. \text{ attesa} + \text{burst time}}$$

46/03/2023 MARZO

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella, e considerato che l'ordine di arrivo dei processi è P1, P2, P3, P4, P5.

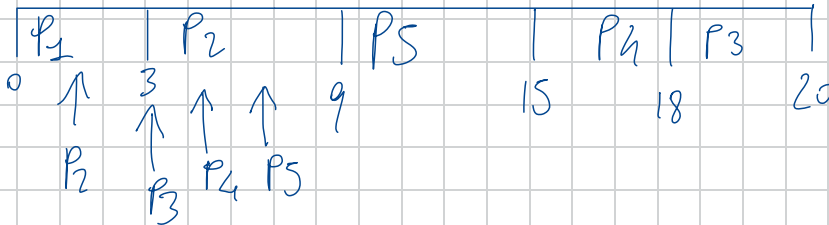
| Processo | Burst Time | Priorità | Tempo di arrivo |
|----------|------------|----------|-----------------|
| P1       | 3          | 3        | 0               |
| P2       | 6          | 1        | 1               |
| P3       | 2          | 3        | 3               |
| P4       | 3          | 2        | 4               |
| P5       | 6          | 1        | 5               |

- a) Disegnare 2 diagrammi di Gantt che illustrino l'esecuzione di questi processi usando gli algoritmi di priorità e round robin (quanto = 3). Indicare l'istante di tempo di ciascuna commutazione fra un processo e l'altro.
- b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo d'attesa e come l'avete applicato per calcolare i risultati esposti)

|    | Priorità | RR |
|----|----------|----|
| P1 |          |    |
| P2 |          |    |
| P3 |          |    |
| P4 |          |    |
| P5 |          |    |

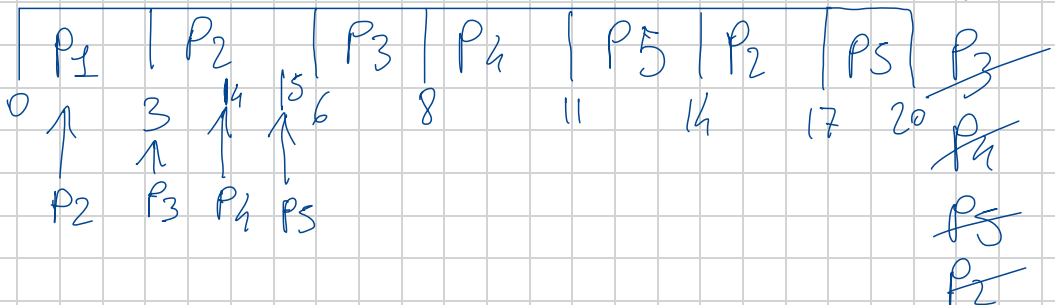
- c) Calcolare il tempo di risposta per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in estrema sintesi definire il tempo di risposta e come l'avete applicato per calcolare i risultati esposti)

Priority



Round Robin  $q=3ms$

Coda



$$t.2^{da} = t. \text{completamento} - t. \text{Arrivo} - BT$$

$$t. \text{risposta} = t. \text{inizio esecuzione} - t. \text{Arrivo}$$

Priority

$$t.2^{da} = P_1(3-0-3) = 0$$

$$P_2(9-1-6) = 2$$

$$P_3(20-3-2) = 15$$

$$P_4(18-4-3) = 11$$

$$P_5(15-5-6) = 4$$

$$t_{\text{risp}} = P_1(0-0) = 0$$

$$P_2(3-1) = 2$$

$$P_3(18-3) = 15$$

$$P_4(15-4) = 11$$

$$P_5(9-5) = 4$$

RR

$$t.2^{da} = P_1(3-0-3) = 0$$

$$P_2(17-1-6) = 10$$

$$P_3(8-3-2) = 3$$

$$P_4(12-4-3) = 4$$

$$P_5(20-5-6) = 9$$

$$t_{\text{risp}} = P_1(0-0) = 0$$

$$P_2(3-1) = 2$$

$$P_3(6-3) = 3$$

$$P_4(8-4) = 4$$

$$P_5(11-5) = 6$$

4. Nel contesto dei metodi per evitare i deadlock:

a) Dati tre thread  $T_0$ ,  $T_1$ ,  $T_2$  e tre tipi di risorsa A (4 istanze), B (3 istanze), C (5 istanze) con le seguenti matrici di

Allocazione e Max:

|       | Allocazione |   |   |   | Max |   |   |
|-------|-------------|---|---|---|-----|---|---|
|       | A           | B | C |   | A   | B | C |
| $T_0$ | 2           | 1 | 2 | . | 3   | 2 | 4 |
| $T_1$ | 1           | 0 | 1 |   | 1   | 2 | 3 |
| $T_2$ | 1           | 1 | 0 |   | 1   | 2 | 2 |

b) calcolare il vettore Disponibile e la matrice Bisogno

c) Verificare se il sistema è in uno stato sicuro (spiegare i passaggi)

d) Può  $T_1$  richiedere (0, 1, 0)? Se viene assegnato cosa succede?

array available  $\Rightarrow$   $\begin{matrix} A & B & C \\ 0 & 1 & 2 \end{matrix}$

CALCOLI

$$A = 4 - (2 + 1 + 1) = 0$$

$$B = 3 - (1 + 0 + 1) = 1$$

$$C = 5 - (2 + 1 + 0) = 2$$

Need (Max - Alloc)

A B C

$t_0$  1 1 2

$t_1$  0 2 2

$t_2$  0 1 2

esiste  $\text{Need} \leq \text{Available}$ ?

work / Available

A B C

0 1 2  $\rightarrow$  start

$t_2 \rightarrow t_1 \rightarrow t_0$

Stato sicuro

1 2 2  $t_2^+ (1 1 0)$

2 2 3  $t_1^+ (1 0 1)$

3 3 5  $t_0 (1 1 2)$

Alloc  $t_1 (0, 1, 0)$  può ri-chiedere?

|       | A | B | C |
|-------|---|---|---|
| $t_0$ | 2 | 1 | 2 |
| $t_1$ | 1 | 1 | 1 |
| $t_2$ | 1 | 1 | 0 |

Available 

| A | B | C |
|---|---|---|
| 0 | 0 | 2 |

$$A = 4 - (2 + 1 + 1) = 0$$

$$B = 3 - (1 + 1 + 1) = 0$$

$$C = 5 - (2 + 1 + 0) = 2$$

Need

|       | A | B | C |
|-------|---|---|---|
| $t_0$ | 1 | 1 | 2 |
| $t_1$ | 0 | 1 | 2 |
| $t_2$ | 0 | 1 | 2 |

|       | Max | A | B | C |
|-------|-----|---|---|---|
| $t_0$ | 3   | 2 | 4 |   |
| $t_1$ | 1   | 2 | 3 |   |
| $t_2$ | 1   | 2 | 2 |   |

esiste un  $t_n$  Need  $\leq$  Allocation?

NO, lo stato non si trova in uno stato sicuro perché non riusciamo a creare una sequenza di processi e quindi a liberare le risorse.

5. Si consideri una memoria paginata:

Si assumano indirizzi logici a 32 bit e pagine da 16 KB

a) Quante entry occorrono per una tabella delle pagine di un solo livello?

b) Quante entry per una tabella delle pagine di secondo livello assumendo 4 KB per la pagina di primo livello?

2)  $\frac{2^{32} \text{ bit indir logico}}{(1024 \cdot 16) \rightarrow 16.384} \rightarrow \frac{2^{32}}{2^{14}} = 2^{18} \text{ entry}$

$\nearrow$   
16 Kb dim pag

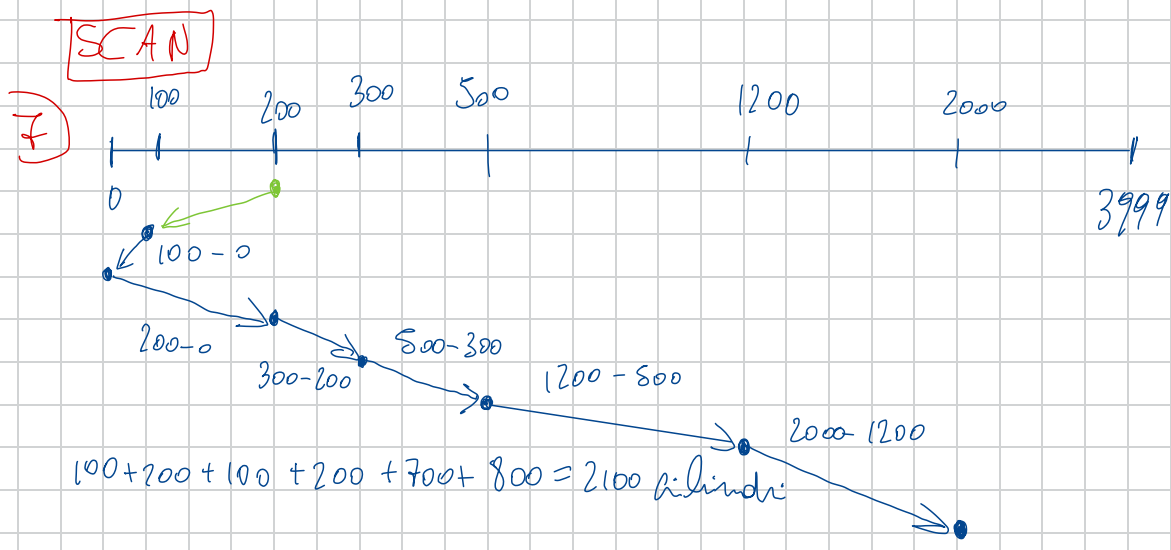
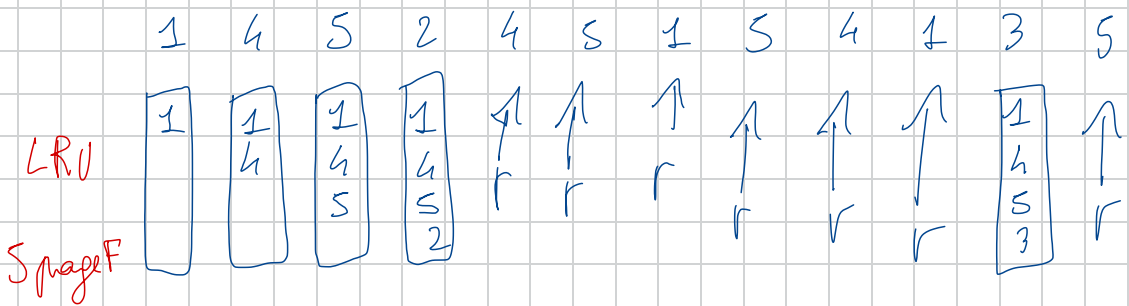
6)  $\frac{2^{18}}{2^{12}} = 2^6 \text{ entry} \quad (4 \cdot 1024 = 2^{12})$



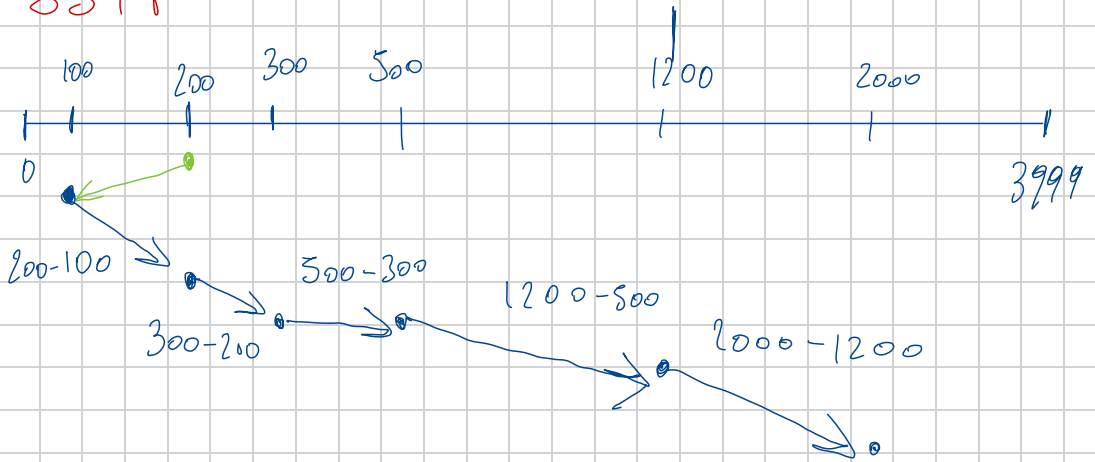
~~X~~ Si consideri un processo che genera la seguente stringa dei riferimenti alle pagine virtuali:  
1, 4, 5, 2, 4, 5, 1, 5, 4, 1, 3, 5

- Se il processo ha 4 frame gestiti con LRU quanti page fault vengono generati?
  - Quanti page fault vengono generati con l'algoritmo FIFO?
  - Spiegare brevemente cos'è la stringa dei riferimenti e a cosa servono gli algoritmi di sostituzione di pagina
7. Assumendo 3000 cilindri e testina su 100 (con richiesta precedente a 200)
- Datta la coda di richieste 2000, 1200, 300, 200, 500 come vengono servite le richieste con l'algoritmo SCAN
  - Qual è la distanza in cilindri percorsi della testina con l'algoritmo SSTF?
  - Spiegare brevemente per quali dispositivi sono utili questi algoritmi e perché

r=refresh la pagina  
indicate diventa la più  
giovane tra i frame, come  
se fosse stata appena inserita



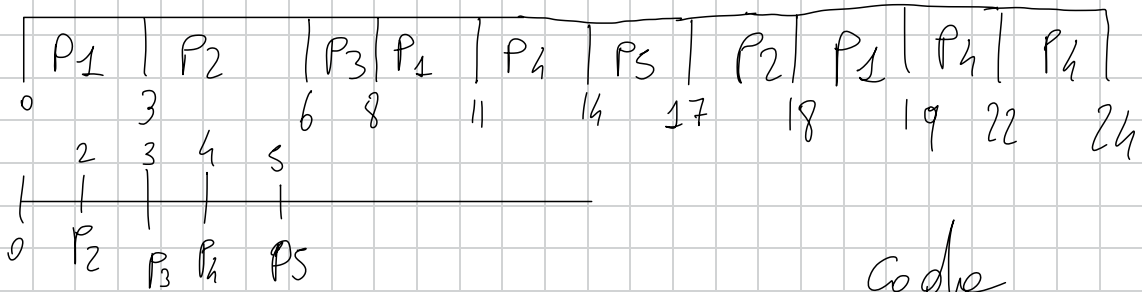
SSTF



$$100 + 100 + 200 + 700 + 800 = 1900 \text{ cilindri}$$

|       |                                                |         |
|-------|------------------------------------------------|---------|
| $P_1$ | BT<br><del>7</del> <del>4</del> <del>1</del> ✓ | AV<br>0 |
| $P_2$ | <del>4</del> <del>1</del> ✓                    | 2       |
| $P_3$ | 2 ✓                                            | 3       |
| $P_4$ | <del>8</del> <del>5</del> 2                    | 4       |
| $P_5$ | <del>3</del> ✓                                 | 5       |

RR  $q=3$



Code

~~$P_2$~~

~~$P_3$~~

~~$P_4$~~

~~$P_5$~~

~~$P_2$~~

~~$P_1$~~

~~$P_4$~~

$P_4$

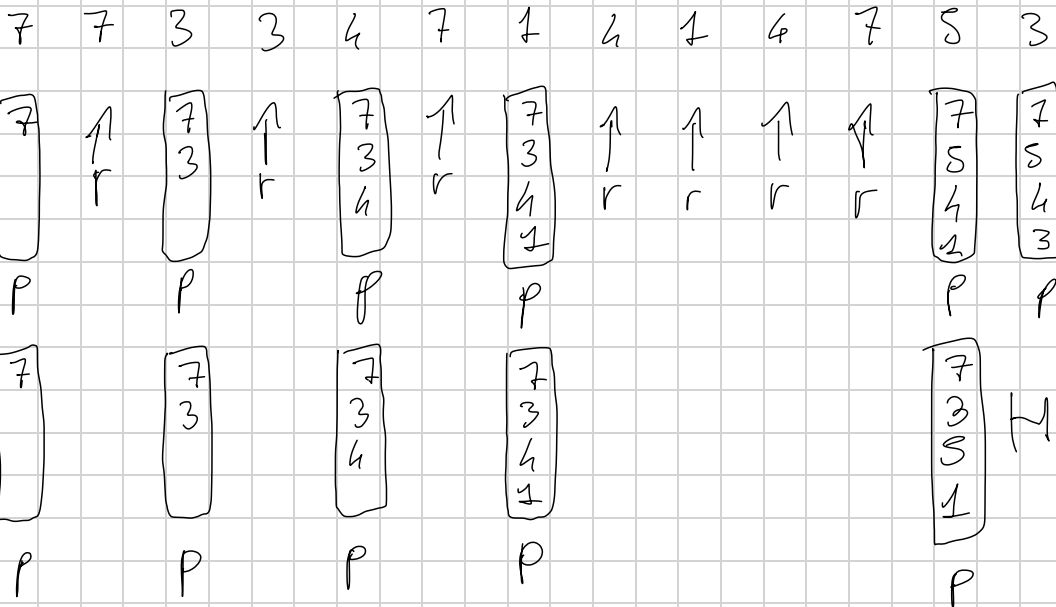
L<sub>1</sub> Frame

LRU

6P

OPT

8P



Nome e cognome Antonio Paoletti Matricola N.86604952

La durata della prova è di 2 ore. Scrivere in stampatello maiuscolo. È obbligatorio restituire il testo e tutti i fogli forniti, anche se non si consegna il compito.

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella, e considerato che l'ordine di arrivo dei processi è P1, P2, P3, P4, P5.

| Processo | Burst Time | Priorità | Tempo di arrivo |
|----------|------------|----------|-----------------|
| P1       | 7          | 3        | 0               |
| P2       | 5          | 1        | 2               |
| P3       | 6          | 3        | 3               |
| P4       | 3          | 4        | 4               |
| P5       | 6          | 1        | 5               |

a) Disegnare 2 diagrammi di Gantt che illustino l'esecuzione di questi processi usando gli algoritmi di scheduling: Priorità (con prelazione e RR, quanto = 5 ms) Indicare l'istante di tempo di ciascuna commutazione tra processi. VEDERE PAG. 10b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in sintesi definire il tempo d'attesa e come l'avete applicato per calcolare i risultati esposti). T<sub>attesa</sub> = T<sub>turn around</sub> - T<sub>turn around</sub> - T<sub>turn around</sub>

|    | RR | Priorità prelazione |
|----|----|---------------------|
| P1 | 18 | 11                  |
| P2 | 3  | 0                   |
| P3 | 17 | 15                  |
| P4 | 11 | 20                  |
| P5 | 16 | 2                   |

T<sub>turn around</sub> = tempo in cui termina l'esecuzione

c) Calcolare il tempo di turnaround per ciascun processo e per ciascun algoritmo di scheduling. Descrivere il procedimento usato (in sintesi definire il tempo di risposta e come l'avete applicato per calcolare i risultati esposti).

|    | RR | Priorità prelazione |
|----|----|---------------------|
| P1 | 0  | 0                   |
| P2 | 2  | 0                   |
| P3 | 17 | 15                  |
| P4 | 11 | 20                  |
| P5 | 13 | 2                   |

T<sub>risposta</sub> = il tempo di tempo da va da quando il processo

2. Tenuto conto che:

sarà, fino all'istante in cui inizia la sua esecuzione

pthread\_create(&amp;threads[0], NULL, func2, NULL);

pthread\_create(&amp;threads[1], NULL, func1, NULL);

pthread\_join(&amp;threads[0], NULL);

pthread\_join(&amp;threads[1], NULL);

pthread\_create(&amp;thread0[0], NULL, func1, NULL);

printf("id %d\n", var1, var2);

return 0;

4. Nel contesto dei metodi per evitare i deadlock:

a) Dati tre thread T1, T2 e tre tipi di risorsa A (4 istanze), B (4 istanze), C (5 istanze) con le seguenti matrici di Allocazione e Max:

|    | Allocazione | Max   | B150640 | C1504016 |
|----|-------------|-------|---------|----------|
|    | A B C       | A B C | A B C   | A B C    |
| T0 | 0 0 1       | 3 3 4 | 0 0 0   | 1 0 2    |
| T1 | 3 1 0       | 3 3 4 | 0 2 4   | 2        |
| T2 | 3 1 2       | 4 3 4 | 1 2 2   |          |

b) Calcolare il vettore Disponibile e la matrice Bisogno

c) Verificare se il sistema è in uno stato sicuro (passaggi)

d) Può T0 richiedere subito (0, 1, 0)? Se viene assegnato lo stato rimane sicuro?

5. Si consideri una memoria paginata:

Si assumano 4096 pagine di indirizzi logici di 4096 pagine con pagine di 8 KB mappate su una memoria fisica da 2048 frame.

a) Quante sono i bit dell'indirizzo logico?  $2^{12}$ b) Quante sono i bit dell'indirizzo fisico?  $2^{11}$ c) Quanti entry avrà una tabella delle pagine?  $2^{11}$ 

6. Si consideri un processo che genera la seguente stringa dei riferimenti alle pagine virtuali:

5, 4, 3, 4, 1, 2, 1, 4, 4, 5, 3, 6, 4

a) Se il processo ha 4 frame gestiti con LRU quanti page fault vengono generati? 8b) Quanti page fault vengono generati con l'algoritmo FIFO? 8

c) Spiegare brevemente che funzione svolge un algoritmo di sostituzione di pagina

7. Assumendo 2000 cilindri e testina su 1200 (con richiesta precedente a 900)

a) Data la coda di richieste 1000, 500, 1650, 200, 30, 450 come vengono servite le richieste con SSTF? quadrato fissob) Qual è la distanza in cilindri per la testina con l'algoritmo SSTF? 2070

c) Spiegare a cosa serve lo scheduling del disco

8. Cos'è un vettore delle interruzioni? Descrivere come viene utilizzato per la gestione delle interruzioni

o il pid del processo padre è 9 e quello del primo figlio è 10 e quello del secondo figlio è 11

Cosa stamperà il seguente programma? Giustificare in estrema sintesi la risposta. Vedere foglio

```

int main()
{
    int var = 1;
    pid_t pid1, pid2;
    printf("Sono il processo padre.\n");
    pid1 = fork();
    var = var + 2;
    if (pid1 == 0)
        var = var + 3;
    else
        pid2 = fork();
        var = var + pid1 + pid2;
}
printf("pid=id var=%d pid1=%d pid2=%d\n",
       getpid(), var, pid1, pid2);
return 0;

```

3. Dato il frammento di programma che segue:

a) Quanti processi e quanti thread genera il frammento di programma seguente? 2 processi, 6 thread

b) Evidenziare, se ci sono, le code critiche e le sezioni critiche

c) Cosa stamperà il programma? 77

Giustificare in estrema sintesi le risposte

```

#define NUM_THREADS 2
int var1 = 1, var2 = 2;
void *func1(void* param)
{
    int a = 3;
    var1 = var1 + a;
    return 0;
}
void *func2(void* param)
{
    int b = 4;
    var2 = b + var2 + var1;
    return 0;
}
int main()
{
    pthread_t threads[NUM_THREADS];
    fork();
}

```

9. Descrivere schematicamente come viene gestito un page fault

10. Descrivere le principali modalità di allocazione dei file evidenziandone vantaggi e svantaggi

24/09/2024 18:19

REDMI NOTE 13 PRO

24/09/2024 18:19

REDMI NOTE 13 PRO

## 2. Tenuto conto che:

- il pid del processo padre è 5 e quello del primo figlio è 6 e del secondo è 7.

pid e ff

Cosa stamperà il seguente programma? Giustificare in estrema sintesi la risposta

```
int var = 5;
```

↓  
getpid()

```
int var = 5;
```

```
int main()  
{
```

```
    pid_t pid1, pid2;  
    int status = 2;  
    printf("Sono il processo padre.\n");
```

```
    pid1 = fork();
```

P  
F<sub>1</sub>

F<sub>1</sub>

```
    status++;
```

```
    if(pid1 == 0) {  
        pid2 = fork();  
        var = var + pid1 + pid2;  
    }
```

F<sub>1</sub> → 6  
F<sub>2</sub>

```
    var = var + status;  
    printf("var = %d \n", var);  
    printf("pid = %d \n", getpid());
```

```
    return 0;
```

P (5)

```
int var = 5;  
int main() {  
    int status = 2;  
    status++ // 3  
    pid1 = 6;  
    if no access {  
        var = 5 + 3 = 8
```

F<sub>1</sub> (6)

```
int var = 5,  
int main() {  
    int status = 2;  
    status++ // 3  
    pid1 = 0  
  
    if (pid == 0) {  
        pid2 = fork();  
        pid2 = 7  
        var = 5 + 0 + 7  
    }  
    var = 12 + 3 = 15
```

F<sub>2</sub> (7)

```
int var = 5,  
int main() {  
    int status = 2;  
    status++ // 3  
    pid1 = 0  
  
    if (pid == 0) {  
        pid2 = fork();  
        pid2 = 0  
        var = 5 + 0 + 0  
    }  
    var = 5 + 3 = 8
```

P(5)

F<sub>1</sub>(6)

F<sub>2</sub>(7)

var = 8

var = 15

var = 8

pid = 5

pid = 6

pid = 7

cos'è il Tempo di risposta = tempo che intercorre tra la sottomissione della richiesta e la prima risposta prodotta, tempo che il sistema impiega nel ricevere e un dato input

deadlock = è una situazione di stallo dei processi, e si verificano grazie a queste condizioni:

Mutua esclusione, quando c'è una risorsa condivisa che può essere utilizzata da un solo processo alla volta, un altro processo richiede tale risorsa bisogna ritardare il processo fino all'rilascio di tale risorsa.

Hold and wait = un proc. che possiede almeno una risorsa aspetta di acquisire risorse già in possesso di altri processi

No preemption = non esiste un diritto di prelazione sulle risorse, vale a dire che le risorse non sono rilasciate dal PROCESSO CHE LE POSSIEDE solo volontariamente, dopo aver terminato il proprio comp. B.

Circular wait: deve esistere un insieme di processi tale che il corrente aspetta la risorsa del successivo.

Need Matx = rappresenta la quantità di risorse aggiuntive che ciascun processo potrebbe richiedere per completare l'esecuzione



Vettore available: rappresenta quantità di risorse attualmente disponibili nel sistema.

turn around = l'intervallo che intercorre tra sottomissione del processo e il completamento dell'esecuzione

lo scheduling del disco a cosa serve?

è un meccanismo che gestisce l'ordine in cui le richieste di accesso vengono servite, importante perché accessi al disco soprattutto se si parla di HDD è molto lento, le testine devono muoversi fisicamente per leggere dei dati.

OBBIETTIVO principale minimizzare i tempi di attesa

spiegare brevemente che funz. svolge l'algoritmo di sostituzione pagine:

è un meccanismo utilizzato dai SO per gestire la memoria virtuale. Quando un programma richiede più memoria di quella fisicamente disponibile dalla RAM, l'SO usa come estensione della memoria una parte del disco, chiamato swap o file paging.

Tuttavia quando la pagina è piena e una nuova pagina di memoria deve essere caricata, l' SO deve decidere quale pagina esistente in RAM deve rimuovere per sostituirla con quella nuova.

LRU sostituisce quella non utilizzata per più tempo

